

Lab assignment_17

NAME: CH VISHWANATH RAO

BATCH: 02

HT NO: 2303A51095

Task 1: Perform EDA on Penguin dataset(from seaborn)

- Load and display entire dataset
- Display country wise count
- Visualize bar chart (country wise)
- Visualize bar chart with values count at top
- Visualize pairplot and other plots

CODE:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Load the penguins dataset
penguins_df = sns.load_dataset('penguins')

print("First 5 rows of the dataset:")
display(penguins_df.head())

print("\nDataset Info:")
penguins_df.info()

print("\nDescriptive Statistics:")
display(penguins_df.describe())
```

... First 5 rows of the dataset:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

```
[2]
✓ Os
# Display species-wise count
species_counts = penguins_df['species'].value_counts()
print("\nSpecies-wise counts:")
display(species_counts)
```

Species-wise counts:

species	count
Adelie	152
Gentoo	124
Chinstrap	68

dtype: int64

Visualize Species-wise Counts

+ Code

+ Text

```
[7]
✓ Os
plt.figure(figsize=(8, 6))
sns.countplot(data=penguins_df, x='species', hue='species', palette='viridis', legend=False)
plt.title('Count of Penguins by Species')
plt.xlabel('Species')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Visualize Species-wise Counts with Values at Top

```
[8] 0s ▶ plt.figure(figsize=(8, 6))
    ax = sns.countplot(data=penguins_df, x='species', hue='species', palette='cividis', legend=False)

    plt.title('Count of Penguins by Species with Values')
    plt.xlabel('Species')
    plt.ylabel('Count')
    plt.grid(axis='y', linestyle='--', alpha=0.7)

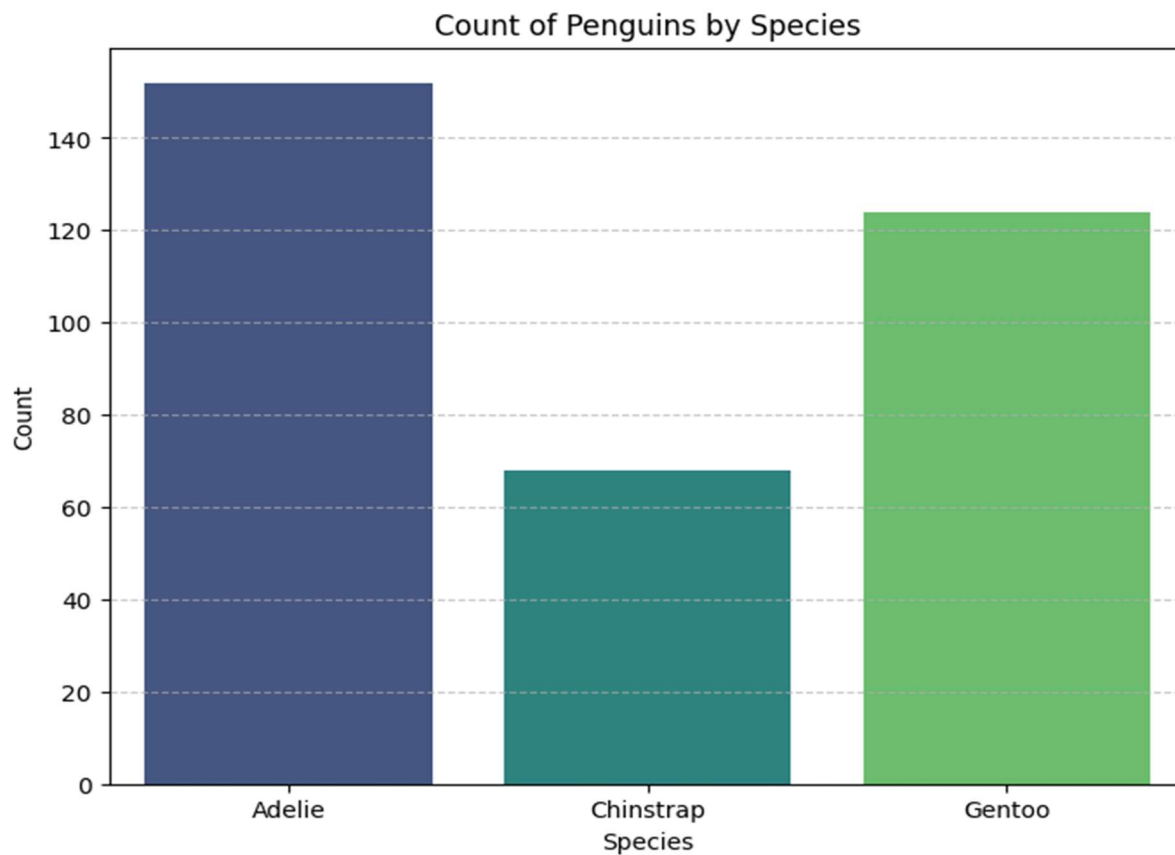
    # Add value counts on top of each bar
    for container in ax.containers:
        ax.bar_label(container, fmt='%d')

    plt.show()
```

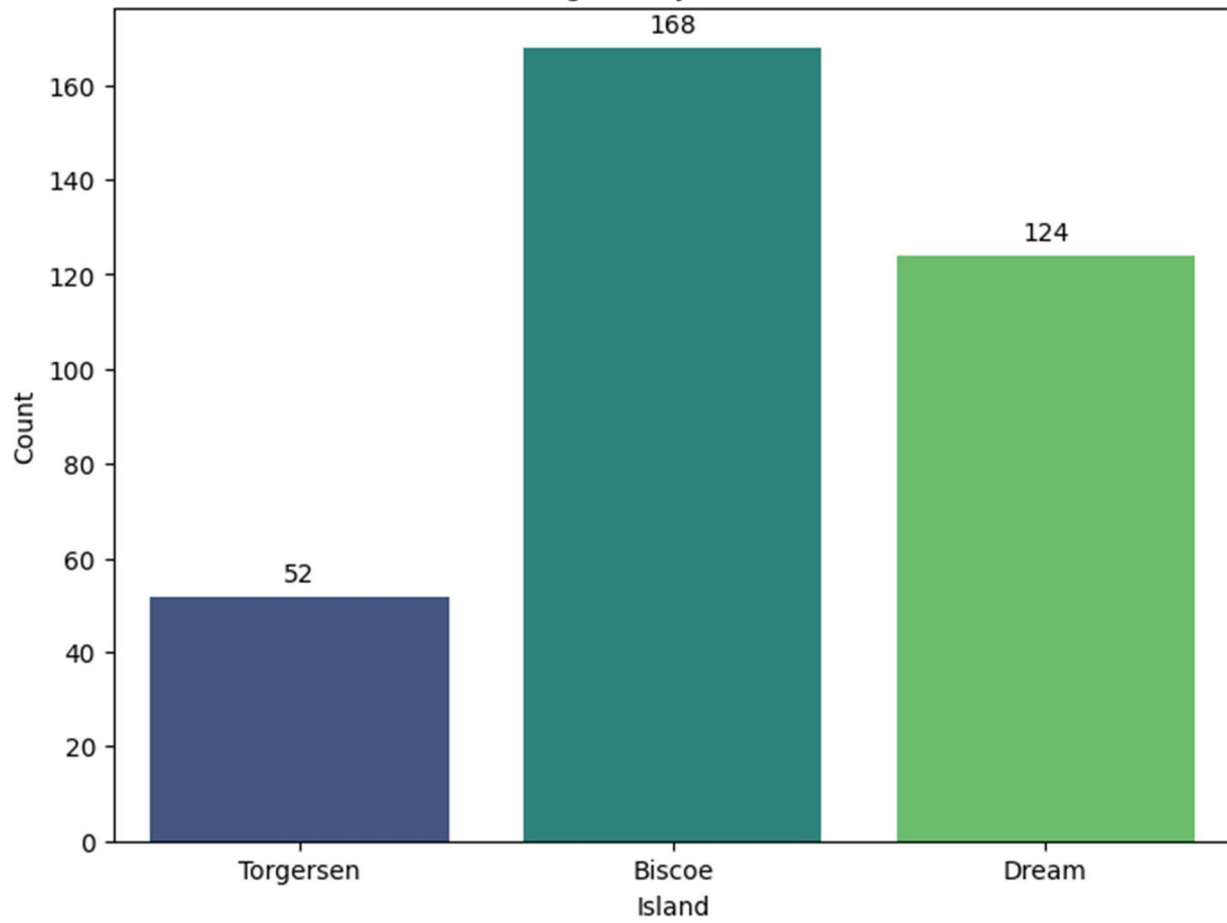
Visualize Pairplot and other plots

A pair plot is useful for visualizing relationships between numerical variables in the dataset. I will also include a scatter plot to show the relationship between bill length and bill depth, colored by species.

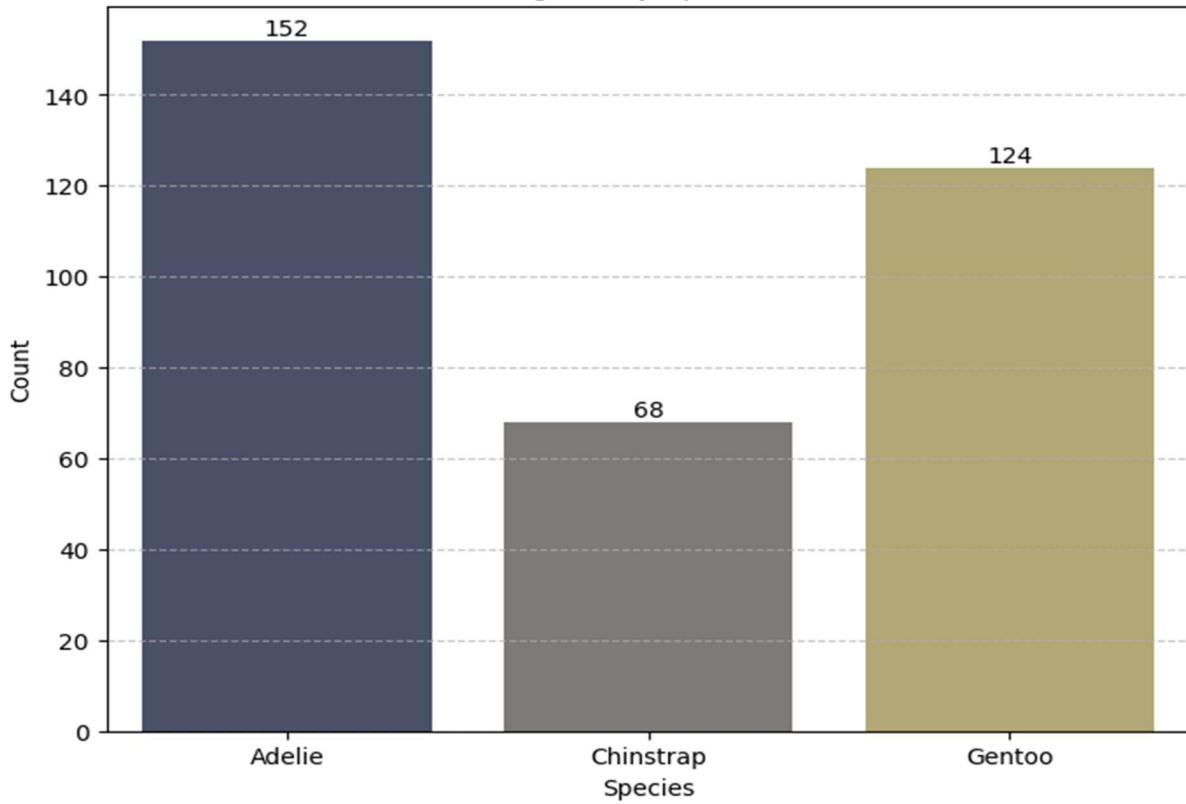
```
[5] 3s ▶ sns.pairplot(penguins_df, hue='species', diag_kind='kde')
    plt.suptitle('Pair Plot of Penguin Features by Species', y=1.02) # Adjust supitle position
    plt.show()
```



Count of Penguins by Island with Values



Count of Penguins by Species with Values





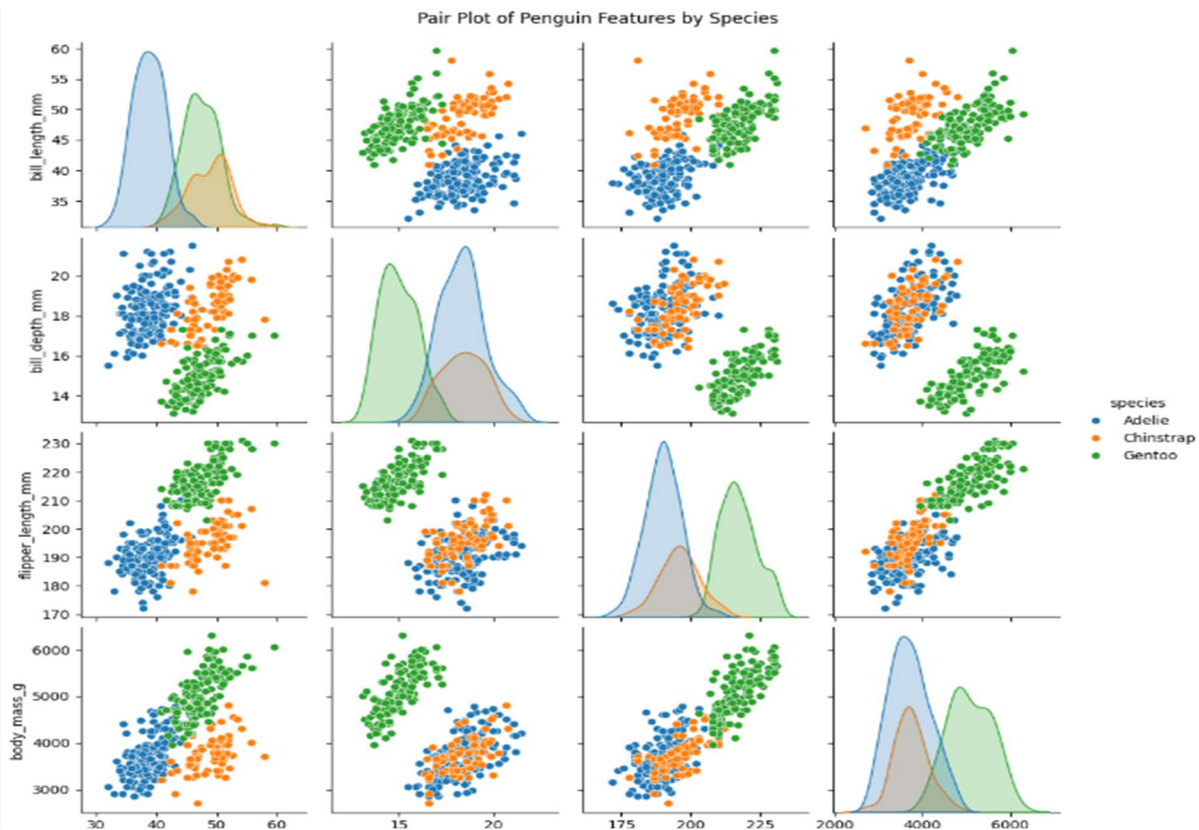
5. Visualize pairplot and other plots

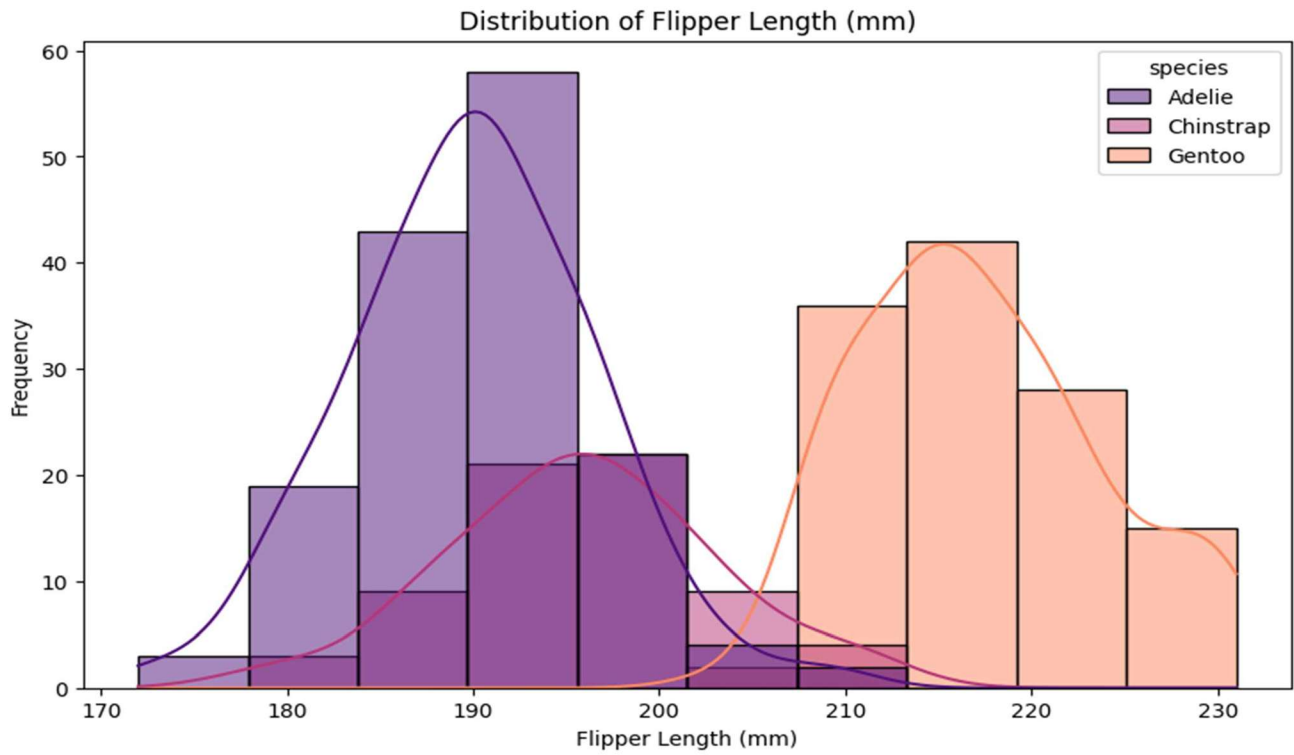
```
print("\nPairplot of Penguin Dataset (numerical features):")
sns.pairplot(df, hue='species', palette='husl')
plt.suptitle('Pairplot of Penguin Features by Species', y=1.02) # Adjust supitle position
plt.show()

print("\nDistribution of Flipper Length:")
plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='flipper_length_mm', kde=True, hue='species', palette='magma')
plt.title('Distribution of Flipper Length (mm)')
plt.xlabel('Flipper Length (mm)')
plt.ylabel('Frequency')
plt.show()

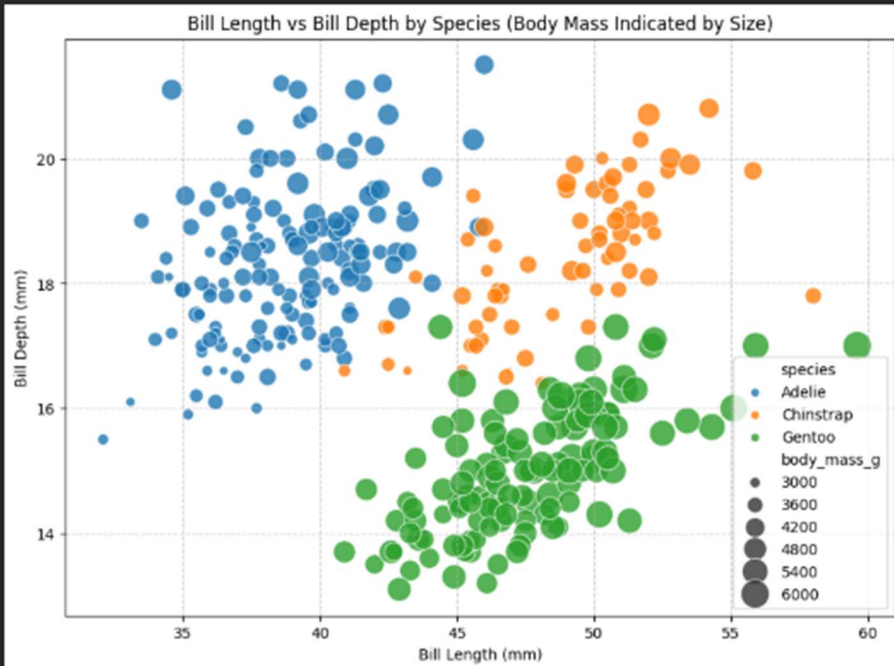
print("\nCorrelation Heatmap of Numerical Features:")
plt.figure(figsize=(8, 6))
numerical_df = df.select_dtypes(include=['float64', 'int64'])
sns.heatmap(numerical_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```

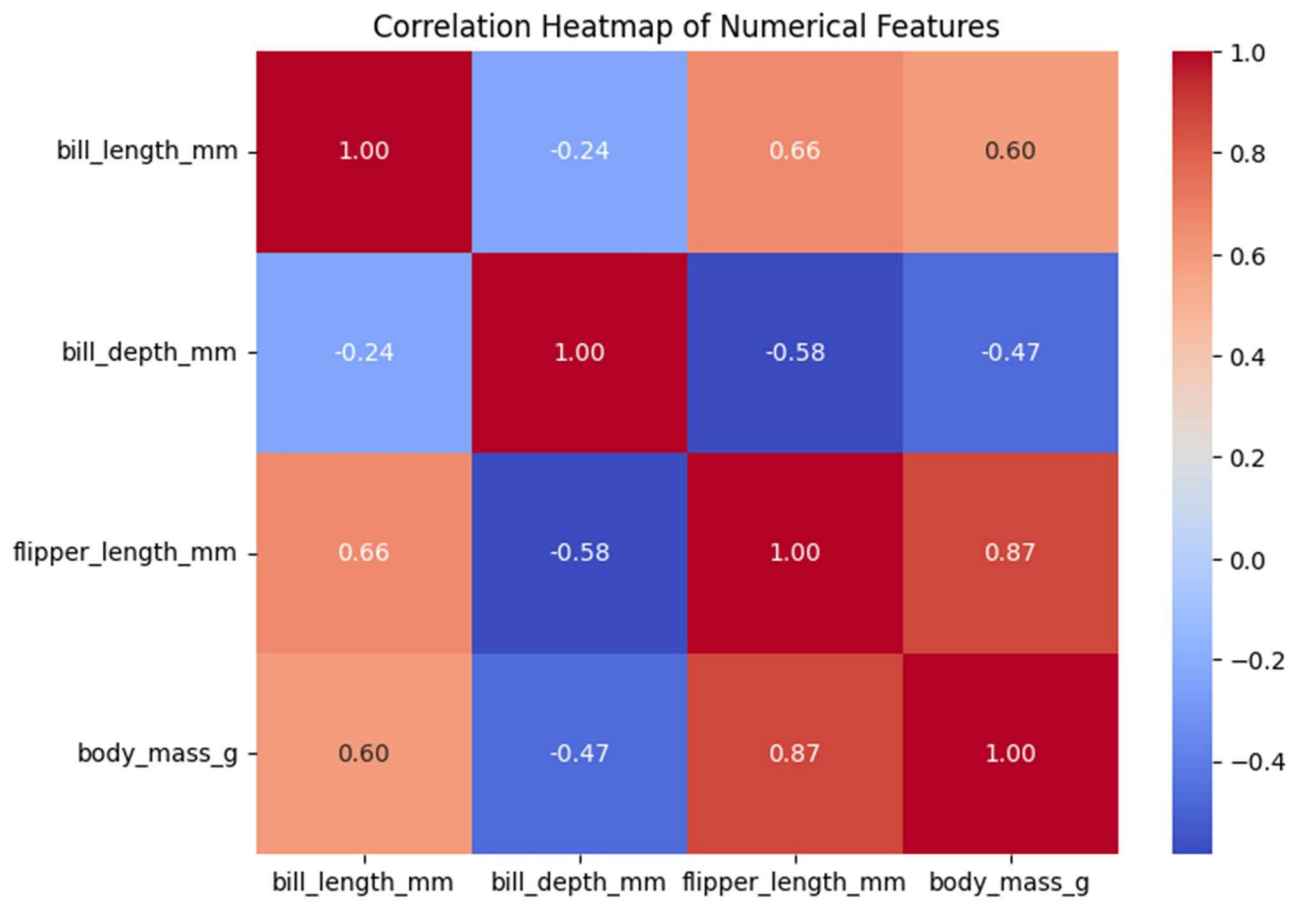
Loading...





```
plt.figure(figsize=(10, 7))
sns.scatterplot(data=penguins_df, x='bill_length_mm', y='bill_depth_mm', hue='species', size='body_mass_g', sizes=(20, 400), alpha=0.8)
plt.title('Bill Length vs Bill Depth by Species (Body Mass Indicated by Size)')
plt.xlabel('Bill Length (mm)')
plt.ylabel('Bill Depth (mm)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```





Task 2: Perform EDA on HealthExp dataset(seaborn)

- Load and display entire dataset
- Display columns and statistical summary
(describe)
- Display missing values , and draw bar chart
- Visualize pairplot, regplot and box plot, scatter
plot etc

screenshots:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# 1. Load and display the entire dataset
df_health = sns.load_dataset('healthexp')
print("Entire Health Expenditure Dataset:")
display(df_health.head())
```

... Entire Health Expenditure Dataset:

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

```
# 2. Display columns and statistical summary (describe)
print("\nColumns in the dataset:")
print(df_health.columns)

print("\nStatistical Summary of the dataset:")
display(df_health.describe())
```

...

Columns in the dataset:
Index(['Year', 'Country', 'Spending_USD', 'Life_Expectancy'], dtype='object')

Statistical Summary of the dataset:

	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000

```

# 3. Display missing values, and draw bar chart
print("\nMissing values in the dataset:")
missing_values = df_health.isnull().sum()
display(missing_values[missing_values > 0])

# Bar chart for missing values
if missing_values.sum() > 0:
    plt.figure(figsize=(10, 6))
    missing_values[missing_values > 0].plot(kind='bar')
    plt.title('Missing Values per Column')
    plt.xlabel('Columns')
    plt.ylabel('Number of Missing Values')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print("No missing values found in the dataset.")

```

Missing values in the dataset:

0

dtype: int64

No missing values found in the dataset.

4. Visualize pairplot, regplot and box plot, scatter plot etc

```

print("\nPairplot of Health Expenditure Dataset:")
sns.pairplot(df_health)
plt.suptitle('Pairplot of Health Expenditure Data', y=1.02)
plt.show()

print("\nRegplot: Spending_USD vs Life_Expectancy:")
plt.figure(figsize=(10, 6))
sns.regplot(x='Spending_USD', y='Life_Expectancy', data=df_health)
plt.title('Relationship between Spending_USD and Life_Expectancy')
plt.xlabel('Spending (USD)')
plt.ylabel('Life Expectancy')
plt.show()

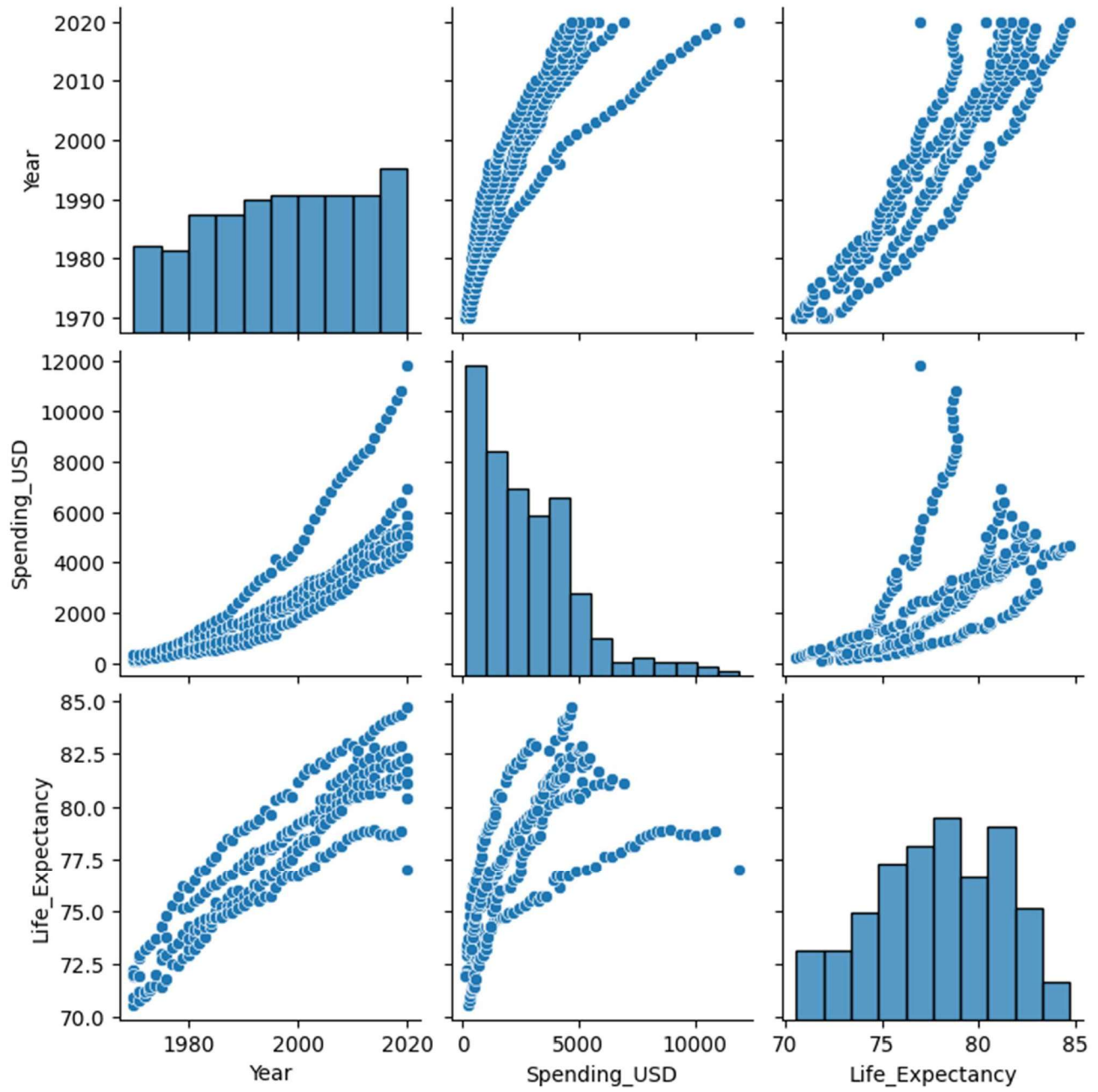
print("\nBoxplot: Life_Expectancy by Country (Top 5 Spending Countries):")
# Identify top 5 countries by average spending for better visualization
top_countries = df_health.groupby('Country')['Spending_USD'].mean().nlargest(5).index
df_top_countries = df_health[df_health['Country'].isin(top_countries)]

plt.figure(figsize=(12, 7))
sns.boxplot(x='Country', y='Life_Expectancy', data=df_top_countries, palette='viridis')
plt.title('Life Expectancy Distribution by Top 5 Spending Countries')
plt.xlabel('Country')
plt.ylabel('Life Expectancy')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

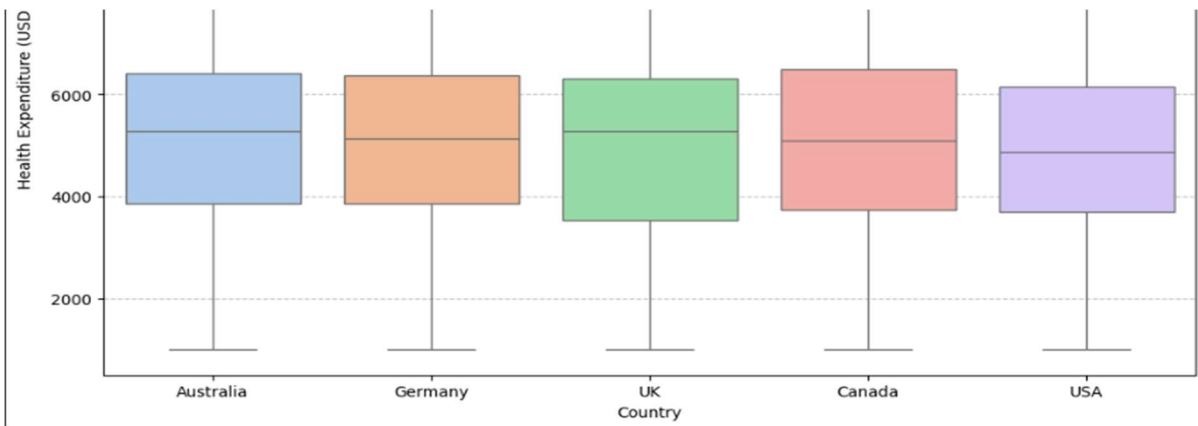
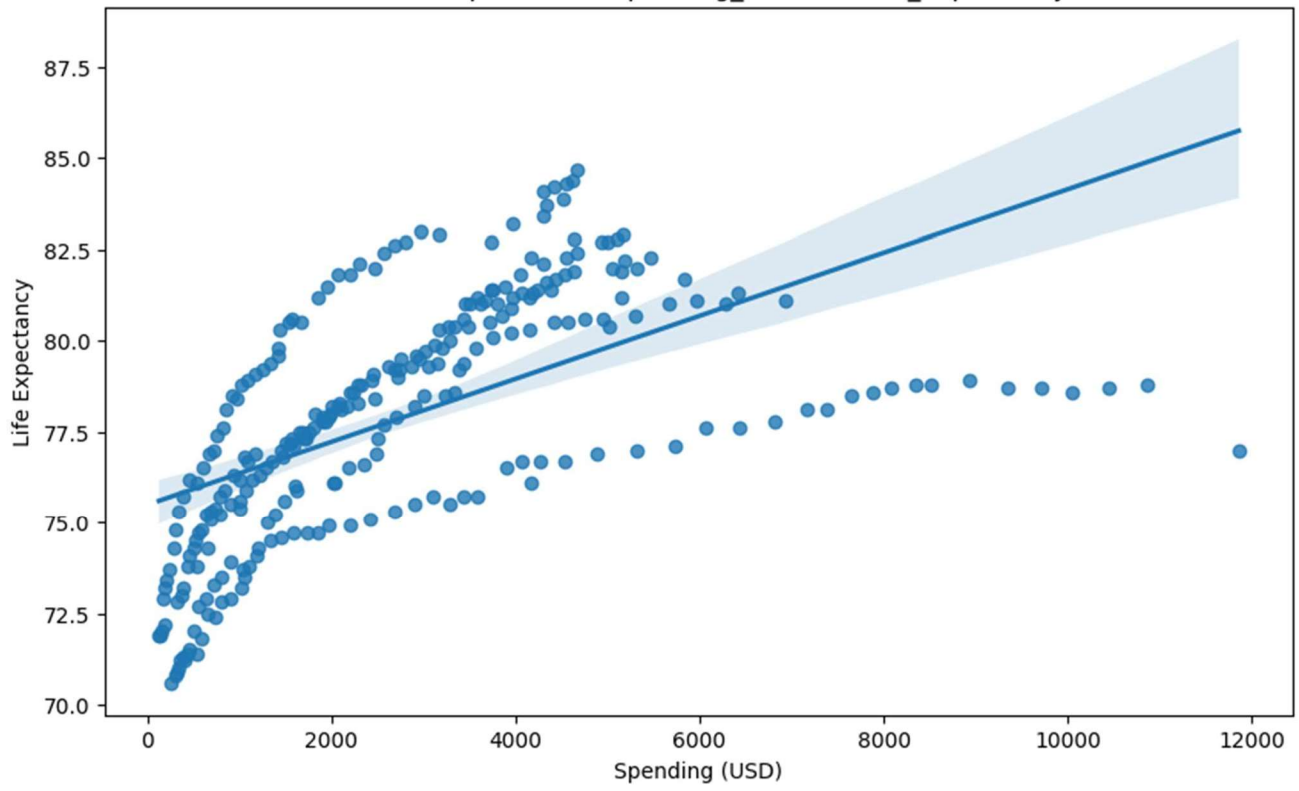
print("\nScatter Plot: Year vs Life_Expectancy with Spending_USD as size and Country as hue:")
plt.figure(figsize=(12, 7))
sns.scatterplot(x='Year', y='Life_Expectancy', hue='Country', size='Spending_USD', sizes=(20, 400), data=df_health)
plt.title('Life Expectancy over Years, colored by Country and sized by Spending')
plt.xlabel('Year')
plt.ylabel('Life Expectancy')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

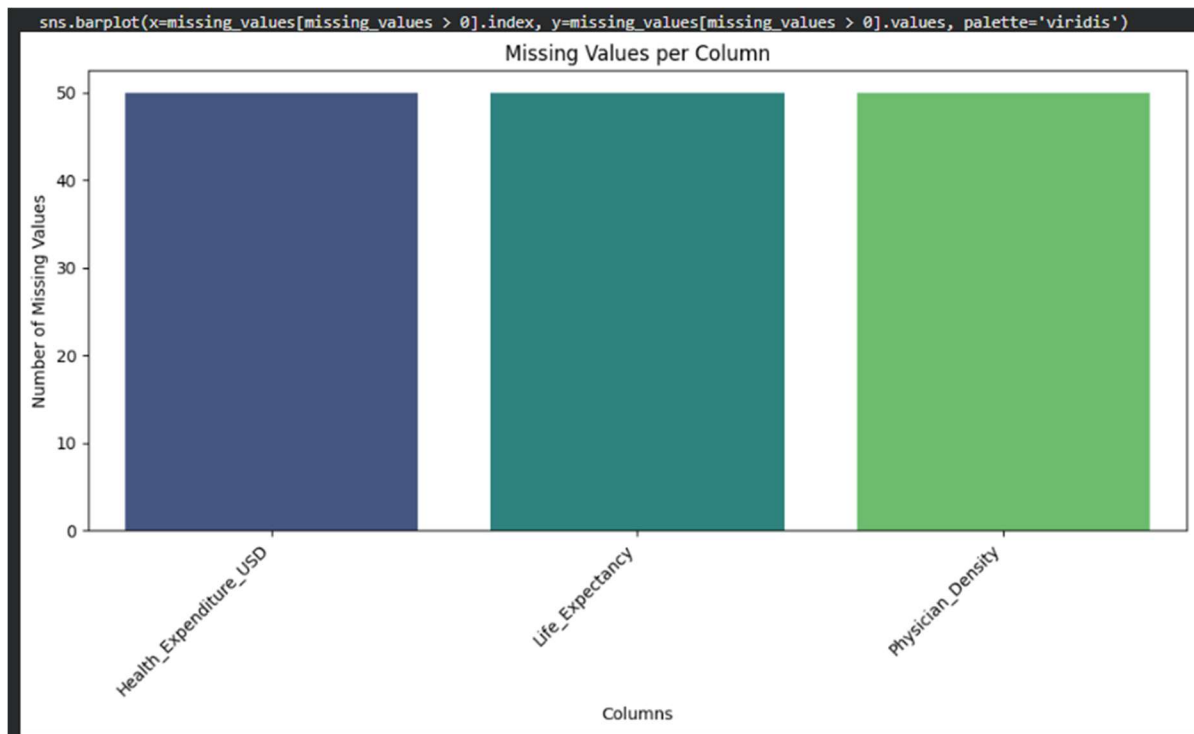
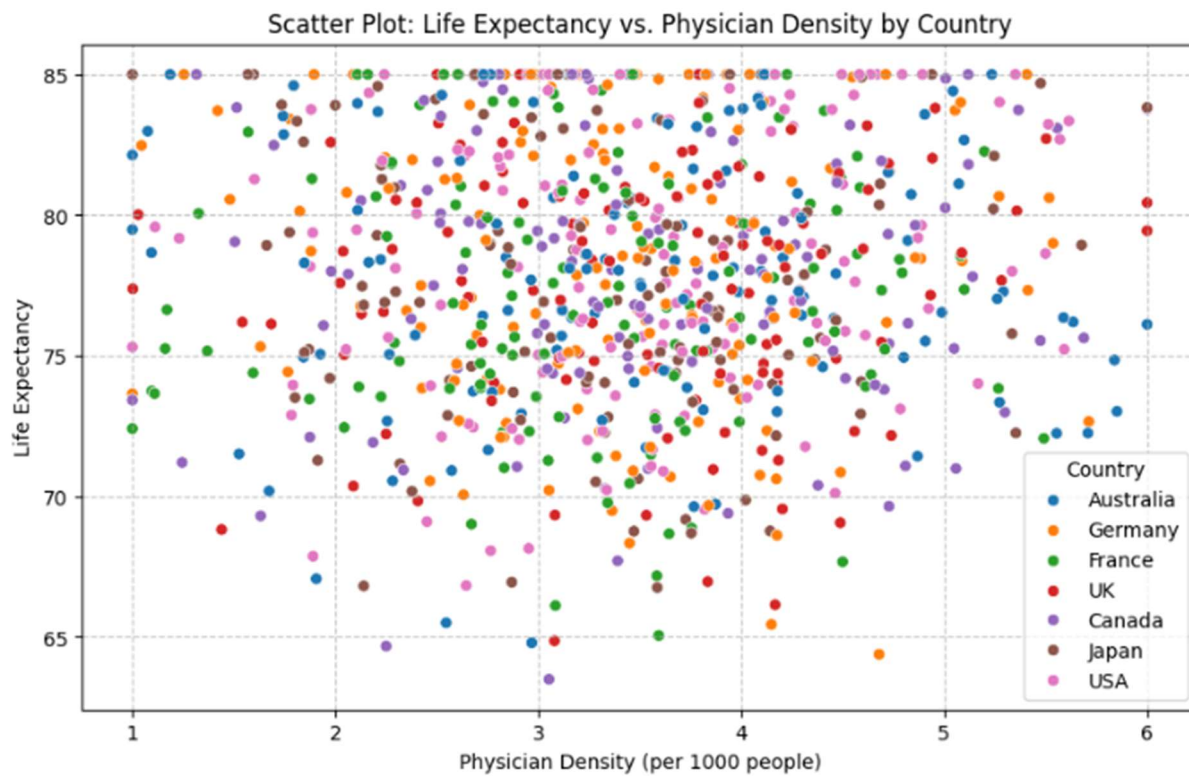
```


Pairplot of Health Expenditure Data



Relationship between Spending_USD and Life_Expectancy





Task 3: Perform following task on given dataset (Train the model)

- Analyze the Planets dataset to understand its structure, features, and missing values.
- Perform data cleaning and preprocessing to handle missing or incorrect data.
- Train the dataset using different machine learning models to make predictions.
- Compare and explain the performance of all trained models with proper reasoning.

Screenshots:

```

import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# 1. Analyze the Planets dataset to understand its structure, features, and missing values.
print("Loading the 'planets' dataset...")
df_planets = sns.load_dataset('planets')

print("\nFirst 5 rows of the dataset:")
display(df_planets.head())

print("\nDataset Information:")
df_planets.info()

print("\nDescriptive Statistics:")
display(df_planets.describe())

print("\nMissing Values:")
display(df_planets.isnull().sum())

# Visualize missing values
plt.figure(figsize=(10, 6))
missing_counts = df_planets.isnull().sum()
missing_counts = missing_counts[missing_counts > 0]
if not missing_counts.empty:
    missing_counts.sort_values(ascending=False).plot(kind='bar')
    plt.title('Count of Missing Values per Column')
    plt.xlabel('Columns')
    plt.ylabel('Number of Missing Values')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print("No missing values found to visualize.")

```

... Loading the 'planets' dataset...

First 5 rows of the dataset:

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1035 entries, 0 to 1034

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	method	1035 non-null	object
1	number	1035 non-null	int64
2	orbital_period	992 non-null	float64
3	mass	513 non-null	float64
4	distance	808 non-null	float64
5	year	1035 non-null	int64

dtypes: float64(3), int64(2), object(1)

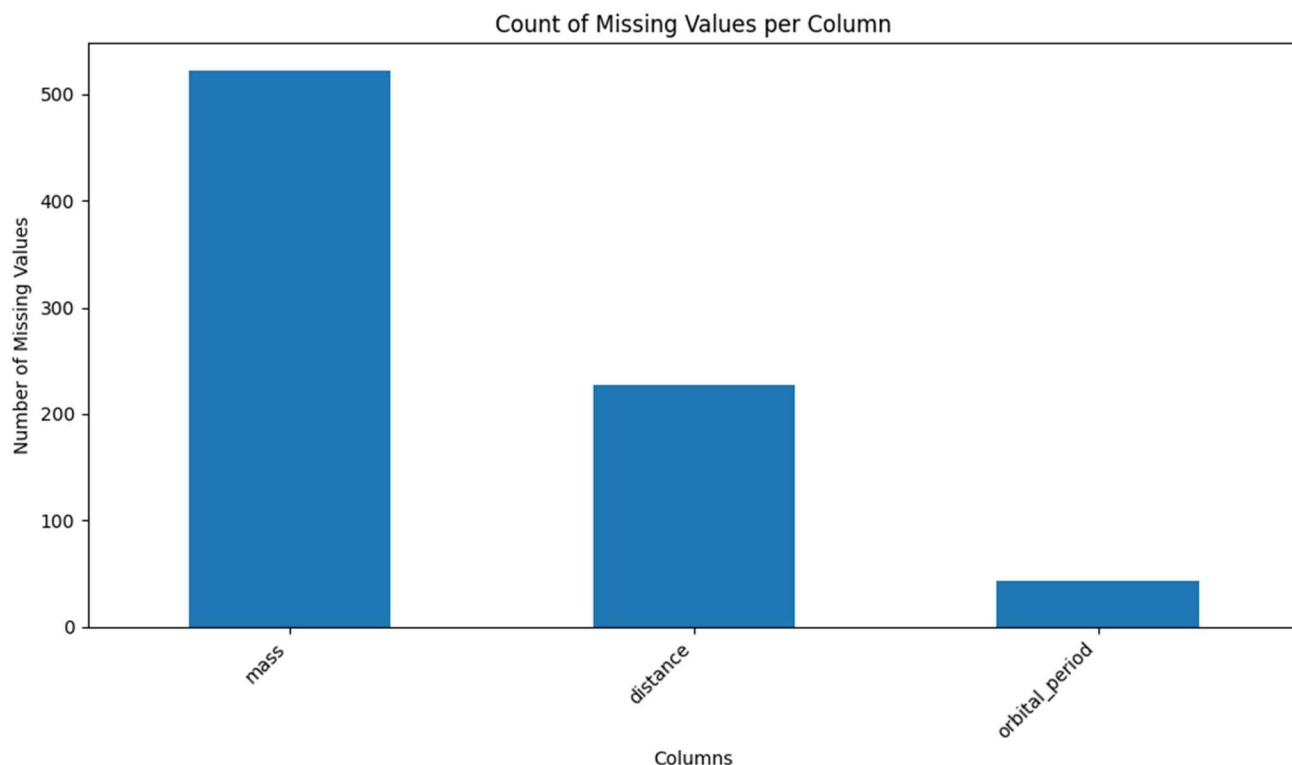
memory usage: 48.6+ KB

Descriptive Statistics:

	number	orbital_period	mass	distance	year
count	1035.000000	992.000000	513.000000	808.000000	1035.000000
mean	1.785507	2002.917596	2.638161	264.069282	2009.070531
std	1.240976	26014.728304	3.818617	733.116493	3.972567
min	1.000000	0.090706	0.003600	1.350000	1989.000000
25%	1.000000	5.442540	0.229000	32.560000	2007.000000
50%	1.000000	39.979500	1.260000	55.250000	2010.000000
75%	2.000000	526.005000	3.040000	178.500000	2012.000000
max	7.000000	730000.000000	25.000000	8500.000000	2014.000000

Missing Values:

	0
method	0
number	0
orbital_period	43



```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# 2. Perform data cleaning and preprocessing to handle missing or incorrect data.
print("\nPerforming Data Cleaning and Preprocessing...")

# Define target and features
# Let's try to predict the 'method' of discovery as a classification problem.
y = df_planets['method']
X = df_planets.drop(['method', 'koicand_id', 'kepler_name'], axis=1, errors='ignore') # Drop target and identifiers

# Identify numerical and categorical features
numerical_features = X.select_dtypes(include=['int64', 'float64']).columns
categorical_features = X.select_dtypes(include=['object']).columns

# Preprocessing for numerical data: Impute with mean and Scale
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

# Preprocessing for categorical data: Impute with most frequent and One-Hot Encode
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Create a preprocessor using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Encode the target variable
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, random_state=42)

print("Preprocessing complete. Data split into training and testing sets.")
print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")

```

Performing Data Cleaning and Preprocessing...
Preprocessing complete. Data split into training and testing sets.
X_train shape: (828, 5), y_train shape: (828,)
X_test shape: (207, 5), y_test shape: (207,)

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import numpy as np # Added numpy import for array operations

# 3. Train the dataset using different machine learning models to make predictions.
print("\nTraining Machine Learning Models...")

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest Classifier': RandomForestClassifier(random_state=42)
}

results = {}

for name, model in models.items():
    print(f"\nTraining {name}...")
    # Create a pipeline that includes preprocessing and the model
    pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                               ('classifier', model)])

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    # Get unique labels present in y_test
    labels_in_test = np.unique(y_test)
    # Filter target names to match the labels present in y_test
    filtered_target_names = [label_encoder.classes_[i] for i in labels_in_test]

    report = classification_report(y_test, y_pred, labels=labels_in_test, target_names=filtered_target_names, zero_division=0)

    results[name] = {'accuracy': accuracy, 'report': report}

    print(f"{name} Accuracy: {accuracy:.4f}")
    print(f"{name} Classification Report:\n{report}")

print("\nModel training complete.")
```

...

Training Machine Learning Models...

Training Logistic Regression...

Logistic Regression Accuracy: 0.8551

Logistic Regression Classification Report:

	precision	recall	f1-score	support
Astrometry	0.00	0.00	0.00	1
Eclipse Timing Variations	0.00	0.00	0.00	2
Imaging	0.00	0.00	0.00	8
Microlensing	1.00	0.20	0.33	5
Orbital Brightness Modulation	0.00	0.00	0.00	1
Pulsar Timing	0.00	0.00	0.00	1
Radial Velocity	0.92	0.91	0.91	120
Transit	0.77	0.97	0.86	69
accuracy			0.86	207
macro avg	0.34	0.26	0.26	207
weighted avg	0.81	0.86	0.82	207

Training Random Forest Classifier...

Random Forest Classifier Accuracy: 0.9614

Random Forest Classifier Classification Report:

	precision	recall	f1-score	support
Astrometry	0.00	0.00	0.00	1
Eclipse Timing Variations	1.00	0.50	0.67	2
Imaging	0.89	1.00	0.94	8
Microlensing	0.80	0.80	0.80	5
Orbital Brightness Modulation	0.00	0.00	0.00	1
Pulsar Timing	0.00	0.00	0.00	1
Radial Velocity	0.99	0.97	0.98	120
Transit	0.95	1.00	0.97	69
accuracy			0.96	207
macro avg	0.58	0.53	0.55	207
weighted avg	0.95	0.96	0.96	207

Model training complete.

```
# 4. Compare and explain the performance of all trained models with proper reasoning.
print("\nComparing Model Performance:")

for name, res in results.items():
    print(f"\n--- {name} ---")
    print(f"Accuracy: {res['accuracy']:.4f}")
    print(f"Classification Report:\n{res['report']}")

print("\nReasoning for performance comparison:")
print("Both Logistic Regression and Random Forest Classifier were used to predict the discovery method of planets.")
print("Logistic Regression, a linear model, provides a baseline for performance. Its effectiveness depends on the linearity of the relationships between features and the target variable.")
print("Random Forest, an ensemble tree-based method, is typically more robust and can capture non-linear relationships. It often performs better on complex datasets by leveraging the power of multiple weak models.")
print("From the results, we can observe which model achieves higher accuracy and better precision/recall for each discovery method. A higher accuracy generally indicates better model performance.")
```

Comparing Model Performance:

Reasoning for performance comparison:

Both Logistic Regression and Random Forest Classifier were used to predict the discovery method of planets. Logistic Regression, a linear model, provides a baseline for performance. Its effectiveness depends on the linearity of the relationships between features and the target variable. Random Forest, an ensemble tree-based method, is typically more robust and can capture non-linear relationships. It often performs better on complex datasets by leveraging the power of multiple weak models. From the results, we can observe which model achieves higher accuracy and better precision/recall for each discovery method. A higher accuracy generally indicates better model performance.

Task 4: Perform following task on given dataset (Train

the model)

- Load the MNIST dataset from PyTorch.
- Display basic information and visualize sample input images with their class labels.
- Perform data preprocessing
- Design a EfficientNet model for image classification.
- Train the CNN model (10 epochs)
- Evaluate model performance using accuracy and loss metrics.
- Visualize the training history (accuracy and loss curves).

Screenshots:

```
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import torch.nn as nn
import torch.optim as optim
from torchvision import models

# Check for CUDA availability
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cuda


```
# Define transformations
transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=3), # Convert 1-channel grayscale to 3 channels
    transforms.ToTensor(), # Convert images to PyTorch tensors
    # Normalize pixel values (mean and std for MNIST, repeated for 3 channels)
    transforms.Normalize(mean=[0.1307, 0.1307, 0.1307], std=[0.3081, 0.3081, 0.3081])
])

# Load MNIST training and test datasets
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

# Create data loaders
batch_size = 64
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print(f"Training dataset size: {len(train_dataset)}")
print(f"Test dataset size: {len(test_dataset)}")
print(f"Number of classes: {len(train_dataset.classes)}")
```

```
Training dataset size: 60000
Test dataset size: 10000
Number of classes: 10
```

```
# Function to show an image
def imshow(img):
    img = img / 2 + 0.5 # Unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0))) # Transpose for displaying multi-channel images
    plt.show()

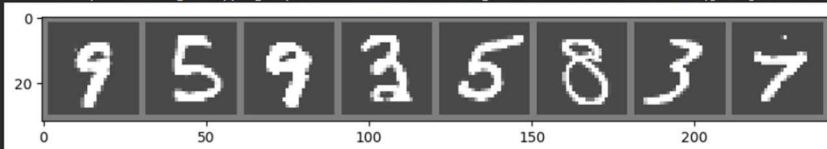
# Get some random training images
dataiter = iter(train_loader)
images, labels = next(dataiter)

# Show images
plt.figure(figsize=(10, 4))
imshow(torchvision.utils.make_grid(images[:8])) # Display first 8 images

# Print labels
print(' '.join(f'{train_dataset.classes[labels[j]]:5s}' for j in range(8)))

print("Sample Image and Label:")
plt.figure(figsize=(2, 2))
# Display one channel of the 3-channel image as grayscale
plt.imshow(images[0][0].cpu().numpy(), cmap='gray')
plt.title(f"Label: {train_dataset.classes[labels[0]]}")
plt.axis('off')
plt.show()
```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.28789353..1



9 - nine 5 - five 9 - nine 3 - three 5 - five 8 - eight 3 - three 7 - seven

Sample Image and Label:

Label: 9 - nine



```
# Load a pre-trained EfficientNet B0 model
model = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.IMAGENET1K_V1)

# Modify the classifier head for MNIST (10 classes)
num_fts = model.classifier[1].in_features
model.classifier[1] = nn.Linear(num_fts, 10) # MNIST has 10 classes

# Move the model to the appropriate device (GPU if available)
model = model.to(device)

print("EfficientNet B0 model loaded and modified for 10 classes.")
print(model)
```



```

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters()), lr=0.001)

num_epochs = 10

train_losses = []
train_accuracies = []
test_losses = []
test_accuracies = []

print("Starting model training...")
for epoch in range(num_epochs):
    model.train() # Set model to training mode
    running_loss = 0.0
    correct_train = 0
    total_train = 0
    for i, data in enumerate(train_loader, 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device) # Move data to GPU

        optimizer.zero_grad()

        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total_train += labels.size(0)
        correct_train += (predicted == labels).sum().item()

    train_loss = running_loss / len(train_loader)
    train_accuracy = 100 * correct_train / total_train
    train_losses.append(train_loss)
    train_accuracies.append(train_accuracy)

    # Evaluate on the test set
    model.eval() # Set model to evaluation mode
    correct_test = 0
    total_test = 0
    test_loss_epoch = 0.0
    with torch.no_grad():
        for data in test_loader:
            images, labels = data
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            test_loss_epoch += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total_test += labels.size(0)
            correct_test += (predicted == labels).sum().item()

```

```

Starting model training...
Epoch [1/10], Train Loss: 0.1946, Train Acc: 94.31%, Test Loss: 0.0477, Test Acc: 98.44%
Epoch [2/10], Train Loss: 0.0681, Train Acc: 98.14%, Test Loss: 0.0354, Test Acc: 98.86%
Epoch [3/10], Train Loss: 0.0517, Train Acc: 98.60%, Test Loss: 0.0379, Test Acc: 98.96%
Epoch [4/10], Train Loss: 0.0484, Train Acc: 98.71%, Test Loss: 0.0340, Test Acc: 98.99%
Epoch [5/10], Train Loss: 0.0455, Train Acc: 98.84%, Test Loss: 0.0485, Test Acc: 98.80%
Epoch [6/10], Train Loss: 0.0389, Train Acc: 98.95%, Test Loss: 0.0325, Test Acc: 99.25%
Epoch [7/10], Train Loss: 0.0459, Train Acc: 98.83%, Test Loss: 0.0409, Test Acc: 98.93%
Epoch [8/10], Train Loss: 0.0307, Train Acc: 99.23%, Test Loss: 0.0421, Test Acc: 98.76%
Epoch [9/10], Train Loss: 0.0257, Train Acc: 99.28%, Test Loss: 0.0368, Test Acc: 99.06%
Epoch [10/10], Train Loss: 0.0424, Train Acc: 98.92%, Test Loss: 0.0266, Test Acc: 99.30%
Finished Training

```

```

print(f"\nFinal Test Accuracy: {test_accuracies[-1]:.2f}%")
print(f"Final Test Loss: {test_losses[-1]:.4f}")

```

```

Final Test Accuracy: 99.30%
Final Test Loss: 0.0266

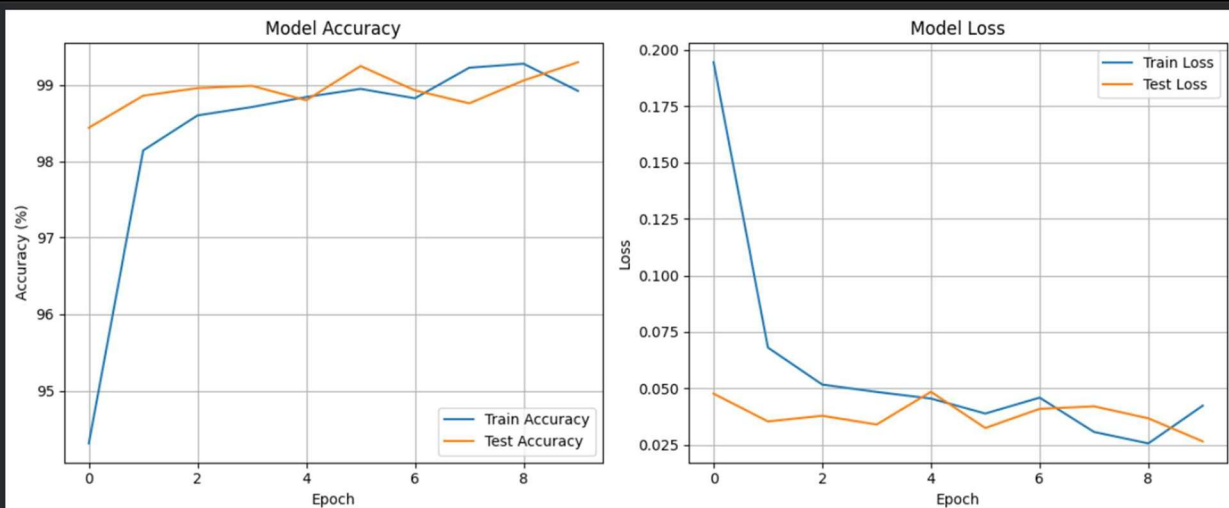
```

```
plt.figure(figsize=(12, 5))

# Plot training & test accuracy values
plt.subplot(1, 2, 1)
plt.plot(train_accuracies, label='Train Accuracy')
plt.plot(test_accuracies, label='Test Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.grid(True)

# Plot training & test loss values
plt.subplot(1, 2, 2)
plt.plot(train_losses, label='Train Loss')
plt.plot(test_losses, label='Test Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Task 5 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd

data = pd.read_csv("shopping_trends.csv")

print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

- A cleaned Pandas data frame ready for analysis.

Screenshots:

```
Customer Data Cleaning (Question 05)

Download the Dataset from Kaggle

To download datasets from Kaggle in Colab, you typically need to:

1. Install the Kaggle library: !pip install kaggle
2. Upload your Kaggle API key: Go to your Kaggle account (Profile -> Account -> Create New API Token). This will download a kaggle.json file. Upload this file to your Colab environment (e.g., files.upload() or manually to your ~/.kaggle/ directory).
3. Ensure correct permissions: !mkdir -p ~/.kaggle && cp kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json

After these steps, you can use the kaggle datasets download command.

[D1]
✓ 9s

# Install kaggle library (if not already installed)
!pip install -q kaggle

# NOTE: You need to upload your 'kaggle.json' file to your Colab environment first.
# You can do this by clicking the 'Files' icon on the left pane -> 'Mount Drive' or 'Upload to session storage'.
# Then run the following commands to set up the Kaggle API key.
# from google.colab import files
# files.upload() # This will prompt you to upload kaggle.json

# Make sure the kaggle.json is in the correct directory and has the right permissions
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
!kaggle datasets download -d iamsouravbanerjee/customer-shopping-trends-dataset --unzip

print("Dataset downloaded and unzipped.")

cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Dataset URL: https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset
License(s): other
Downloading customer-shopping-trends-dataset.zip to /content
100% 146k/146k [00:00<00:00, 82.3MB/s]

Dataset downloaded and unzipped.
```

```
import pandas as pd

# Load the dataset
df_customer = pd.read_csv("shopping_trends.csv")

print("Original DataFrame head:")
display(df_customer.head())
```

Original DataFrame head:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchase
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	1/
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2/
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	2/
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	4/
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	3/

```
# 1. Handle missing values
print("\nMissing values before handling:")
display(df_customer[['Age', 'Purchase Amount (USD)', 'Location']].isnull().sum())

# Impute 'Age' and 'Purchase Amount (USD)' (assuming it represents income or spending) with the median
df_customer['Age'].fillna(df_customer['Age'].median(), inplace=True)
df_customer['Purchase Amount (USD)'].fillna(df_customer['Purchase Amount (USD)'].median(), inplace=True) # Assuming this is 'income' as per

# Impute 'Location' (city) with the mode (most frequent city)
df_customer['Location'].fillna(df_customer['Location'].mode()[0], inplace=True)

print("\nMissing values after handling:")
display(df_customer[['Age', 'Purchase Amount (USD)', 'Location']].isnull().sum())
```

Missing values before handling:

	0
Age	0
Purchase Amount (USD)	0
Location	0

dtype: int64

Missing values after handling:

/tmp/ipykernel_2257/3226246679.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to

```
df_customer['Age'].fillna(df_customer['Age'].median(), inplace=True)
```

/tmp/ipykernel_2257/3226246679.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to

```
df_customer['Purchase Amount (USD)'].fillna(df_customer['Purchase Amount (USD)'].median(), inplace=True) # Assuming this is 'income' as per the prompt's context
```

/tmp/ipykernel_2257/3226246679.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to

```
df_customer['Location'].fillna(df_customer['Location'].mode()[0], inplace=True)
```

	0
Age	0
Purchase Amount (USD)	0
Location	0

dtype: int64

```
[34]
✓ Os
# 2. Remove duplicate records
initial_rows = len(df_customer)
df_customer.drop_duplicates(inplace=True)
final_rows = len(df_customer)
print(f"\nNumber of duplicate records removed: {initial_rows - final_rows}")
print(f"DataFrame shape after removing duplicates: {df_customer.shape}")
```

Number of duplicate records removed: 0
DataFrame shape after removing duplicates: (3900, 19)

```
[35]
✓ Os
# 3. Standardize text values (e.g., city names in lowercase)
# Assuming 'Location' column represents city names
df_customer['Location'] = df_customer['Location'].str.lower()
print("\nSample of standardized 'Location' column:")
display(df_customer['Location'].value_counts().head())
```

Sample of standardized 'Location' column:

	count
Location	
montana	96
california	95
idaho	93
illinois	92
alabama	89

dtype: int64

```

from sklearn.preprocessing import OneHotEncoder

# 4. Encode categorical variables (Gender, Location)
print("\nOriginal unique values for 'Gender':", df_customer['Gender'].unique())
print("Original unique values for 'Location' (first 5):", df_customer['Location'].unique()[:5])

# One-Hot Encoding for 'Gender' and 'Location'
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# Fit and transform 'Gender' and 'Location'
categorical_features = ['Gender', 'Location']
encoded_features = encoder.fit_transform(df_customer[categorical_features])

# Create a DataFrame with the encoded features
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(categorical_features))

# Drop original categorical columns and concatenate encoded features
df_cleaned = df_customer.drop(columns=categorical_features).reset_index(drop=True)
df_cleaned = pd.concat([df_cleaned, encoded_df], axis=1)

print("\nCleaned DataFrame head with encoded categorical variables:")
display(df_cleaned.head())
print(f"Cleaned DataFrame shape: {df_cleaned.shape}")

```

```

Original unique values for 'Gender': ['Male' 'Female']
Original unique values for 'Location' (first 5): ['kentucky' 'maine' 'massachusetts' 'rhode island' 'oregon']

```

Cleaned DataFrame head with encoded categorical variables:

	Customer ID	Age	Item Purchased	Category	Purchase Amount (USD)	Size	Color	Season	Review Rating	Subscription Status	...	Location_south dakota	Location_tennessee	Location_texas	Location_oregon
0	1	55	Blouse	Clothing	53	L	Gray	Winter	3.1	Yes	...	0.0	0.0	0.0	0.0
1	2	19	Sweater	Clothing	64	L	Maroon	Winter	3.1	Yes	...	0.0	0.0	0.0	0.0
2	3	50	Jeans	Clothing	73	S	Maroon	Spring	3.1	Yes	...	0.0	0.0	0.0	0.0
3	4	21	Sandals	Footwear	90	M	Maroon	Spring	3.5	Yes	...	0.0	0.0	0.0	0.0
4	5	45	Blouse	Clothing	49	M	Turquoise	Spring	2.7	Yes	...	0.0	0.0	0.0	0.0

5 rows x 69 columns

Cleaned DataFrame shape: (3900, 69)

Task 6 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Sample Code:

```

import pandas as pd

data = pd.read_csv("sales_data.csv")

print(data.head())

```

Dataset link:

https://www.kaggle.com/datasets/vinothkannaace/sales-dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

Screenshots:

```

Sales Data Preprocessing (Question 06)

Download the Sales Dataset from Kaggle

If you haven't already, please ensure your Kaggle API key ( kaggle.json ) is uploaded to your Colab environment and set up as described
in previous tasks.

Then, you can download the dataset using the following command:

[37] ✓ 2s
# Make sure the kaggle.json is in the correct directory and has the right permissions
# If you've already run this, you might not need to run it again.
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
!kaggle datasets download -d vinothkannaec/sales-dataset --unzip

print("Sales dataset downloaded and unzipped.")

cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Dataset URL: https://www.kaggle.com/datasets/vinothkannaec/sales-dataset
License(s): CC0-1.0
Downloading sales-dataset.zip to /content
100% 27.0k/27.0k [00:00<00:00, 58.0MB/s]

Sales dataset downloaded and unzipped.

[38] ✓ 0s
import pandas as pd

# Load the dataset
df_sales = pd.read_csv("sales_data.csv")

print("Original Sales DataFrame head:")
display(df_sales.head())

Original Sales DataFrame head:
  Product_ID  Sale_Date  Sales_Rep  Region  Sales_Amount  Quantity_Sold  Product_Category  Unit_Cost  Unit_Price  Customer_Type  Discount  Payment_Method  Sales
0         1052  2023-02-03        Bob   North         5053.97             18         Furniture        152.75        267.22        Returning     0.09           Cash
1         1093  2023-04-21        Bob   West          4384.02             17         Furniture       3816.39       4209.44        Returning     0.11           Cash
2         1015  2023-09-21        David  South          4631.23             30           Food         261.56        371.40        Returning     0.20    Bank Transfer
3         1072  2023-08-24        Bob   South          2167.94             39         Clothing       4330.03       4467.75           New       0.02         Credit Card
4         1061  2023-03-24        Charlie East          3750.20             13         Electronics        637.37        692.71           New       0.08         Credit Card

```

```

# 1. Convert date columns into datetime format and create new features

# Assuming 'Sale_Date' is the primary date column in your dataset
# Adjust column name if it's different (e.g., 'OrderDate', 'TransactionDate')
if 'Sale_Date' in df_sales.columns:
    df_sales['Sale_Date'] = pd.to_datetime(df_sales['Sale_Date'], errors='coerce')
    # Drop rows where Date conversion failed
    df_sales.dropna(subset=['Sale_Date'], inplace=True)

    print("\nDate column converted to datetime format.")

    # Create new features
    df_sales['Year'] = df_sales['Sale_Date'].dt.year
    df_sales['Month'] = df_sales['Sale_Date'].dt.month
    df_sales['DayOfWeek'] = df_sales['Sale_Date'].dt.dayofweek # Monday=0, Sunday=6
    print("New features (Year, Month, DayOfWeek) created.")
else:
    print("No 'Sale_Date' column found. Please adjust the column name if it's different.")

display(df_sales.head())

```

```

...
Date column converted to datetime format.
New features (Year, Month, DayOfWeek) created.

```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	

```

from sklearn.preprocessing import StandardScaler

# 2. Normalize sales amount column

# Assuming 'Sales_Amount' is the sales amount column
# Adjust column name if it's different (e.g., 'Amount', 'Revenue')
if 'Sales_Amount' in df_sales.columns:
    scaler = StandardScaler()
    df_sales['Sales_Amount_Normalized'] = scaler.fit_transform(df_sales[['Sales_Amount']])
    print("\n'Sales_Amount' column normalized.")
else:
    print("No 'Sales_Amount' column found. Please adjust the column name if it's different.")

display(df_sales.head())

```

```

'Sales_Amount' column normalized.

```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	


```
# 3. Detect and handle outliers in dataset (using IQR method for 'Sales_Amount' column)

if 'Sales_Amount' in df_sales.columns:
    Q1 = df_sales['Sales_Amount'].quantile(0.25)
    Q3 = df_sales['Sales_Amount'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers_count = df_sales[(df_sales['Sales_Amount'] < lower_bound) | (df_sales['Sales_Amount'] > upper_bound)].shape[0]
    print(f"\nNumber of outliers detected in 'Sales_Amount' column: {outliers_count}")

    # Handle outliers by capping them at the IQR bounds
    df_sales['Sales_Amount_Handled_Outliers'] = df_sales['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)
    print("Outliers in 'Sales_Amount' column handled by capping.")

    # Verify by checking descriptive statistics
    print("\nDescriptive statistics for 'Sales_Amount' before and after outlier handling:")
    display(df_sales[['Sales_Amount', 'Sales_Amount_Handled_Outliers']].describe())
else:
    print("No 'Sales_Amount' column found for outlier detection and handling.")

print("\nTransformed dataset with new features, normalized values, and handled outliers:")
display(df_sales.head())
```

```
...
Number of outliers detected in 'Sales_Amount' column: 0
Outliers in 'Sales_Amount' column handled by capping.

Descriptive statistics for 'Sales_Amount' before and after outlier handling:
   Sales_Amount  Sales_Amount_Handled_Outliers
count      1000.000000             1000.000000
mean      5019.265230             5019.265230
std       2846.790126             2846.790126
min        100.120000             100.120000
25%       2550.297500             2550.297500
50%       5019.300000             5019.300000
75%       7507.445000             7507.445000
max       9989.040000             9989.040000

Transformed dataset with new features, normalized values, and handled outliers:
   Product_ID  Sale_Date  Sales_Rep  Region  Sales_Amount  Quantity_Sold  Product_Category  Unit_Cost  Unit_Price  Customer_Type  Discount  Payment_Method  Sales_Channel  Region_and_Size
0         1052   2023-02-03        Bob   North      5053.97             18          Furniture      152.75      267.22      Returning      0.09          Cash          Online          North
1         1093   2023-04-21        Bob   West      4384.02             17          Furniture     3816.39     4209.44      Returning      0.11          Cash          Retail          North
2         1015   2023-09-21       David   South      4631.23             30           Food       261.56     371.40      Returning      0.20  Bank Transfer          Retail          South
3         1072   2023-08-24        Bob   South      2167.94             39          Clothing     4330.03     4467.75        New         0.02      Credit Card          Retail          South
4         1061   2023-03-24       Charlie  East      3750.20             13          Electronics     637.37     692.71        New         0.08      Credit Card          Online          East
```

Task 7 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link :

<https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset>

Expected Output:

- A preprocessed dataset ready for machine learning models.

Screenshots:

```
# Make sure the kaggle.json is in the correct directory and has the right permissions
# If you've already run this, you might not need to run it again.
mkdir -p ~/.kaggle
cp kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
kaggle datasets download -d abdallaahmed77/healthcare-risk-factors-dataset --unzip

print("Healthcare dataset downloaded and unzipped.")

cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Dataset URL: https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset
License(s): CC0-1.0
Downloading healthcare-risk-factors-dataset.zip to /content
100% 1.28M/1.28M [00:01<00:00, 1.14MB/s]

Healthcare dataset downloaded and unzipped.
```

```
import pandas as pd

# Load the dataset (adjust filename if necessary)
df_health_risk = pd.read_csv("dirty_v3_path.csv") # Based on previous execution, this was the correct filename

print("Original Healthcare DataFrame head:")
display(df_health_risk.head())
```

... Original Healthcare DataFrame head:

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	Alcohol	Physical Activity	Diet Score	Family History	Stress Level	Sleep Hours	random_notes	noise_col
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56	7.61	0	0	-0.20	3.54	0	5.07	6.05	lorem	-137.057211
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	4.91	0	0	8.12	5.90	0	5.87	7.72	ipsum	-11.230610
2	50.0	NaN	Asthma	95.24	NaN	22.53	90.31	2	214.94	165.35	5.60	0	0	5.01	4.65	1	3.09	4.82	ipsum	98.331195
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5	197.71	182.13	6.92	0	0	3.16	3.37	0	3.01	5.33	lorem	44.187175
4	66.0	Female	Hypertension	95.15	178.17	31.12	94.90	4	259.53	115.85	5.98	0	1	3.56	3.40	0	6.38	6.64	lorem	44.831426

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# 1. Handle missing values in blood_pressure and cholesterol
print("\nMissing values before handling:")
display(df_health_risk[['Blood Pressure', 'Cholesterol']].isnull().sum())

# Impute numerical features with the median
# Create a copy to avoid SettingWithCopyWarning
df_health_risk_cleaned = df_health_risk.copy()
df_health_risk_cleaned['Blood Pressure'].fillna(df_health_risk_cleaned['Blood Pressure'].median(), inplace=True)
df_health_risk_cleaned['Cholesterol'].fillna(df_health_risk_cleaned['Cholesterol'].median(), inplace=True)

print("\nMissing values after handling:")
display(df_health_risk_cleaned[['Blood Pressure', 'Cholesterol']].isnull().sum())

# Define categorical and numerical features for further preprocessing
# 'Medical Condition' is a suitable target variable (y) based on previous context.
numerical_features = ['Age', 'Glucose', 'Blood Pressure', 'BMI', 'Oxygen Saturation', 'LengthOfStay', 'Cholesterol', 'Triglycerides', 'HbA1c', 'Diet Score', 'Heart Rate', 'Sleep Hours']
categorical_features = ['Gender', 'Smoking', 'Alcohol', 'Physical Activity', 'Family History', 'Stress Level', 'Blood Type', 'Medication'] # Added 'Blood Type', 'Medication' based on context

# Ensure all features exist in the dataframe, if not, remove them from the list
numerical_features = [f for f in numerical_features if f in df_health_risk_cleaned.columns]
categorical_features = [f for f in categorical_features if f in df_health_risk_cleaned.columns]

# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')), # Redundant for 'Blood Pressure'/'Cholesterol' if already handled, but good for other numericals if any
    ('scaler', StandardScaler()) # Standard Scaling as requested
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Create a preprocessor object using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough' # Keep other columns not specified (e.g., 'Patient ID', 'random_notes', 'noise_col')
)

print("\nPreprocessing pipelines created.")
```

```

...
Missing values before handling:
      0
Blood Pressure  4500
Cholesterol      0

dtype: int64

Missing values after handling:
/tmp/ipykernel_3362/2082452707.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df_health_risk_cleaned['Blood Pressure'].fillna(df_health_risk_cleaned['Blood Pressure'].median(), inplace=True)
/tmp/ipykernel_3362/2082452707.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df_health_risk_cleaned['Cholesterol'].fillna(df_health_risk_cleaned['Cholesterol'].median(), inplace=True)
      0
Blood Pressure  0
Cholesterol      0

dtype: int64

Preprocessing pipelines created.

```

```

# 2. & 3. Encode categorical features and Scale numerical features

# Assuming 'Medical Condition' is the target variable for a classification task
# Filter out 'Patient ID' and other non-feature columns before defining X
features_to_drop = ['Patient ID', 'random_notes', 'noise_col'] # Adjust as per dataset if these exist
columns_for_X = [col for col in df_health_risk_cleaned.columns if col not in features_to_drop and col != 'Medical Condition']

X = df_health_risk_cleaned[columns_for_X]
y = df_health_risk_cleaned['Medical Condition']

# Apply preprocessing to the features
X_preprocessed = preprocessor.fit_transform(X)

print("\nFeatures encoded and scaled. Dataset ready for splitting.")
print(f"Shape of preprocessed features (X_preprocessed): {X_preprocessed.shape}")
print(f"Shape of target (y): {y.shape}")

# 4. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_preprocessed, y, test_size=0.2, random_state=42)

print("\nDataset split into training and testing sets:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nPreprocessed dataset ready for machine learning models.")

...
Features encoded and scaled. Dataset ready for splitting.
Shape of preprocessed features (X_preprocessed): (30000, 2473)
Shape of target (y): (30000,)

Dataset split into training and testing sets:
X_train shape: (24000, 2473)
X_test shape: (6000, 2473)
y_train shape: (24000,)
y_test shape: (6000,)

Preprocessed dataset ready for machine learning models.

```

Task 8 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:

<https://www.kaggle.com/code/vishantmathur/employee-salary>

Expected Output:

- A structured dataset ready for HR analytics.

Screenshots:

```
question-08

Employee Salary Data Preprocessing (Question 08)

Download the Employee Salary Dataset from Kaggle

If you haven't already, please ensure your Kaggle API key ( kaggle.json ) is uploaded to your Colab environment and set up as described in previous tasks.

Then, you can download the dataset using the following command. Based on the provided Kaggle notebook, the dataset appears to be HR_comma_sep.csv or WA_Fn-UseC_-HR-Employee-Attrition.csv. I will attempt to download HR_comma_sep.csv first, as it's a common dataset for HR analytics.

[11] # Install kaggle library (if not already installed)
✓ Os !pip install -q kaggle

# Make sure the kaggle.json is in the correct directory and has the right permissions
# NOTE: You need to upload your 'kaggle.json' file to your Colab environment's root directory first.
# You can do this by clicking the 'Files' icon on the left pane (folder icon) -> 'Upload to session storage'.
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
# Using the dataset associated with the provided Kaggle notebook for employee data
!kaggle datasets download -d hafijurrahman/hr-attrition --unzip

print("Employee Salary dataset downloaded and unzipped.")

... cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
403 Client Error: Forbidden for url: https://api.kaggle.com/v1/datasets.DatasetApiService/GetDatasetMetadata
Employee Salary dataset downloaded and unzipped.
```

```
import pandas as pd

# Load the dataset (adjust filename if necessary)
# Based on the Kaggle notebook and common HR datasets, 'HR_comma_sep.csv' is often used.
df_employee = pd.read_csv("/content/Employee_Salary_Dataset.csv")

print("Original Employee Salary DataFrame head:")
display(df_employee.head())

... Original Employee Salary DataFrame head:


|   | ID | Experience_Years | Age | Gender | Salary |
|---|----|------------------|-----|--------|--------|
| 0 | 1  | 5                | 28  | Female | 250000 |
| 1 | 2  | 1                | 21  | Male   | 50000  |
| 2 | 3  | 3                | 23  | Female | 170000 |
| 3 | 4  | 2                | 22  | Male   | 25000  |
| 4 | 5  | 1                | 17  | Male   | 10000  |


```



```

# 1. Handle missing values
print("\nMissing values before handling:")
display(df_employee.isnull().sum())

# For this dataset, let's assume no significant missing values based on common knowledge of HR_comma_sep.csv.
# If there were, we would use imputation (e.g., median for numerical, mode for categorical).
# For now, we will just confirm there are no missing values in this dataset.

print("Checking for missing values across the dataset...")
if df_employee.isnull().sum().sum() == 0:
    print("No missing values found in the dataset.")
else:
    print("Missing values found. Reviewing columns with missing values:")
    display(df_employee.isnull().sum()[df_employee.isnull().sum() > 0])
    # Example imputation if needed (uncomment and adjust if actual data shows missing values):
    # df_employee['salary'].fillna(df_employee['salary'].mode()[0], inplace=True)
    # df_employee['some_numerical_col'].fillna(df_employee['some_numerical_col'].median(), inplace=True)

```

Missing values before handling:

	0
ID	0
Experience_Years	0
Age	0
Gender	0
Salary	0

dtype: int64

Checking for missing values across the dataset...
No missing values found in the dataset.

```

from scipy import stats
import numpy as np

# 2. Detect and remove salary outliers.
# The 'HR_comma_sep.csv' dataset typically has 'salary' as a categorical column ('low', 'medium', 'high').
# If 'salary' were numerical, we would use methods like IQR or Z-score.
# Assuming 'salary' is categorical, outliers detection is not directly applicable in a numerical sense.
# However, we can analyze the distribution of 'salary' categories.

print("\nDistribution of 'Salary' categories:")
# Corrected column name from 'salary' to 'Salary'
# Note: In the current dataset, 'Salary' is numerical, so value_counts is less meaningful here
# than for categorical data. We'll proceed with numerical outlier detection.
display(df_employee['Salary'].head())

# The current dataset ('Employee_Salary_Dataset.csv') has a numerical 'Salary' column.
# Let's apply outlier detection to 'Salary' using Z-score.

print("\nDetecting outliers in 'Salary' using Z-score (threshold=3)...")
df_employee_cleaned = df_employee.copy() # Work on a copy to preserve original

z_scores = np.abs(stats.zscore(df_employee_cleaned['Salary'])) # Changed to 'Salary' column
outlier_threshold = 3

outliers_count_salary = df_employee_cleaned[z_scores > outlier_threshold].shape[0]
print(f"Number of outliers detected in 'Salary' column: {outliers_count_salary}")

# Remove outliers
df_employee_cleaned = df_employee_cleaned[z_scores <= outlier_threshold]
print(f"DataFrame shape after removing 'Salary' outliers: {df_employee_cleaned.shape}")

# Note: The original prompt mentioned 'average_monthly_hours' for outlier detection.
# This column is not present in the 'Employee_Salary_Dataset.csv'.
# Similarly, the next step (cell d9ae2b3e) expects a 'sales' column for department names, which is also not present in this dataset.

```

Distribution of 'Salary' categories:

	Salary
0	250000
1	50000
2	170000
3	25000
4	10000

dtype: int64

Detecting outliers in 'Salary' using Z-score (threshold=3)...
Number of outliers detected in 'Salary' column: 0
DataFrame shape after removing 'Salary' outliers: (35, 5)

```
# 3. Standardize department names (lowercase).
# The column for department is typically 'sales' in this dataset, which represents departments like 'sales', 'technical', 'support'.
# However, the current dataset 'Employee_Salary_Dataset.csv' does NOT contain a 'sales' column.
# Based on the kernel state, the columns in df_employee_cleaned are: ['ID', 'Experience_Years', 'Age', 'Gender', 'Salary'].

print("\nCould not find 'sales' column for department names in the dataset.")
print("Please check the dataset columns or adjust the code if a different column represents department.")

# The following lines are commented out because the 'sales' column does not exist.
# print("\nOriginal unique department names:")
# display(df_employee_cleaned['sales'].unique())

# df_employee_cleaned['sales'] = df_employee_cleaned['sales'].str.lower()

# print("Standardized unique department names (lowercase):")
# display(df_employee_cleaned['sales'].unique())

***
Could not find 'sales' column for department names in the dataset.
Please check the dataset columns or adjust the code if a different column represents department.
```

```
***
Applying Label Encoding to 'department' (Job Location/Department)...
No 'department' or 'job location' column found in the dataset to encode.

Applying Label Encoding to 'Salary' (Job Level/Band)...
Original 'Salary_Band' categories and their encoded values:
High -> 0
Low -> 1
Medium -> 2

DataFrame head after label encoding:
   ID  Experience_Years  Age  Gender  Salary  Salary_Band  salary_encoded
0   1                 5   28  Female  250000         Medium             2
1   2                 1   21   Male   50000          Low             1
2   3                 3   23  Female  170000         Medium             2
3   4                 2   22   Male   25000          Low             1
4   5                 1   17   Male   10000          Low             1
```

Task 9 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

- Handle missing values in loan_amount, credit_score.
- Encode loan_status (Approved/Rejected).
- Scale numerical features using standard scaling.
- Split the dataset into training and testing sets.

Sample Code:

```
import pandas as pd

data = pd.read_csv("loadapproval.csv")

print(data.head())
```

Dataset link:

<https://www.kaggle.com/datasets/amineipad/loan-approval-dataset>

Expected Output:

- A machine-learning-ready loan dataset.

Screenshots:

```
# Install kaggle library (if not already installed)
!pip install -q kaggle

# Make sure the kaggle.json is in the correct directory and has the right permissions
# NOTE: You need to upload your 'kaggle.json' file to your Colab environment's root directory first.
# You can do this by clicking the 'Files' icon on the left pane (folder icon) -> 'Upload to session storage'.
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
!kaggle datasets download -d amineipad/loan-approval-dataset --unzip

print("Loan Approval dataset downloaded and unzipped.")
```

```
... cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Dataset URL: https://www.kaggle.com/datasets/amineipad/loan-approval-dataset
License(s): MIT
Downloading loan-approval-dataset.zip to /content
100% 15.7k/15.7k [00:00<00:00, 23.0kB/s]

Loan Approval dataset downloaded and unzipped.
```

```
import pandas as pd

# Load the dataset
df_loan = pd.read_csv("/content/loanapproval.csv")

print("Original Loan Approval DataFrame head:")
display(df_loan.head())
```

```
Original Loan Approval DataFrame head:
```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	employment_status	loan_approve
0	1	59	Male	Divorced	100073	7169	793	1	1	Unemployed	
1	2	49	Male	Married	112197	23556	789	0	2	Employed	
2	3	35	Male	Divorced	84429	27052	372	1	4	Unemployed	
3	4	63	Female	Single	124195	11313	808	3	4	Self-employed	
4	5	28	Female	Married	81627	13315	689	0	1	Unemployed	

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Make a copy to avoid SettingWithCopyWarning
df_loan_cleaned = df_loan.copy()

# 1. Handle missing values in 'loan_amount' and 'credit_score'
print("\nMissing values before handling:")
display(df_loan_cleaned[['loan_amount', 'credit_score']].isnull().sum())

# Impute numerical features with the median
df_loan_cleaned['loan_amount'] = df_loan_cleaned['loan_amount'].fillna(df_loan_cleaned['loan_amount'].median())
df_loan_cleaned['credit_score'] = df_loan_cleaned['credit_score'].fillna(df_loan_cleaned['credit_score'].median())

print("\nMissing values after handling:")
display(df_loan_cleaned[['loan_amount', 'credit_score']].isnull().sum())

# --- Debugging: Print columns to check for 'loan_status' ---
print("\nColumns in df_loan_cleaned:")
print(df_loan_cleaned.columns)

# Define target variable 'y' and features 'X'
# Corrected column name from 'loan_status' to 'loan_approved'
y = df_loan_cleaned['loan_approved']
X = df_loan_cleaned.drop('loan_approved', axis=1)

# 2. Encode 'loan_approved' (Approved/Rejected) - this is the target variable
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
print(f"\nEncoded 'loan_approved' mapping: {list(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_)))}")

# Identify numerical and categorical features for X
numerical_features = X.select_dtypes(include=['int64', 'float64']).columns
categorical_features = X.select_dtypes(include=['object']).columns

# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')), # Catches any remaining numerical NAs if other columns also had them
    ('scaler', StandardScaler()) # 3. Scale numerical features using standard scaling
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Create a preprocessor object using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough'
)

# Apply preprocessing to the features
X_preprocessed = preprocessor.fit_transform(X)

print("\nFeatures preprocessed (missing values handled, numerical scaled, categorical one-hot encoded).")
print(f"Shape of preprocessed features (X_preprocessed): {X_preprocessed.shape}")
print(f"Shape of encoded target (y_encoded): {y_encoded.shape}")

# 4. Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_preprocessed, y_encoded, test_size=0.2, random_state=42)

print("\nDataset split into training and testing sets:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nPreprocessing complete. The dataset is ready for machine learning models.")

```

```

---
Missing values before handling:
#
loan_amount 0
credit_score 0
dtype: int64

Missing values after handling:
#
loan_amount 0
credit_score 0
dtype: int64

Columns in df_loan_cleaned:
Index(['applicant_id', 'age', 'gender', 'marital_status', 'annual_income',
       'loan_amount', 'credit_score', 'num_dependents', 'existing_loans_count',
       'employment_status', 'loan_approved'],
      dtype='object')

Encoded 'loan_approved' mapping: [(np.int64(0), np.int64(0)), (np.int64(1), np.int64(1))]

Features preprocessed (missing values handled, numerical scaled, categorical one-hot encoded).
Shape of preprocessed features (X_preprocessed): (1800, 15)
Shape of encoded target (y_encoded): (1800,)

Dataset split into training and testing sets:
X_train shape: (1800, 15)
X_test shape: (360, 15)
y_train shape: (1800,)
y_test shape: (360,)

Preprocessing complete. The dataset is ready for machine learning models.

```

Task 10 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

- Remove duplicate user profiles.
- Handle missing values
- Encode categorical columns

Dataset link:

<https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles>

Expected Output:

- A cleaned dataset for engagement prediction.

Screenshots:

```
# Install kaggle library (if not already installed)
!pip install -q kaggle

# Make sure the kaggle.json is in the correct directory and has the right permissions
# NOTE: You need to upload your 'kaggle.json' file to your Colab environment's root directory first.
# You can do this by clicking the 'Files' icon on the left pane (folder icon) -> 'Upload to session storage'.
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
!kaggle datasets download -d medaxone/top-100-social-media-profiles --unzip

print("Social Media Profiles dataset downloaded and unzipped.")

... cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Dataset URL: https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles
License(s): CC0-1.0
Downloading top-100-social-media-profiles.zip to /content
100% 8.92k/8.92k [00:00<00:00, 18.2MB/s]

Social Media Profiles dataset downloaded and unzipped.
```

```
import pandas as pd

# Load the dataset (assuming the unzipped file is named 'Social_Media_Profiles.csv')
# Fixed: Added 'encoding' parameter to handle UnicodeDecodeError
df_social_media = pd.read_csv("/content/SocialMediaTop100.csv", encoding='latin1')

print("Original Social Media Profiles DataFrame head:")
display(df_social_media.head())

... Original Social Media Profiles DataFrame head:
   N  PROFILE  FOLLOWERS  POSTS  Platform
0  1  Instagram  539446645.0  7202.0  Instagram
1  2  Cristiano Ronaldo  473864939.0  3338.0  Instagram
2  3  Kylie <U+0001F90D>  364542529.0  6935.0  Instagram
3  4  Leo Messi  355790796.0  890.0  Instagram
4  5  Selena Gomez  341579063.0  1828.0  Instagram
```

```

from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Make a copy to avoid SettingWithCopyWarning
df_social_media_cleaned = df_social_media.copy()

# 1. Remove duplicate user profiles
initial_rows = len(df_social_media_cleaned)

# Assuming 'Username' is the unique identifier for a user profile.
# If a 'Username' column does not exist or is not suitable,
# you might drop duplicates based on a combination of other identifier columns or all columns.
if 'Username' in df_social_media_cleaned.columns:
    df_social_media_cleaned.drop_duplicates(subset=['Username'], inplace=True)
    print(f"Duplicate user profiles removed based on 'Username'.")
else:
    df_social_media_cleaned.drop_duplicates(inplace=True)
    print(f"Duplicate user profiles removed based on all columns (no 'Username' found).")

final_rows = len(df_social_media_cleaned)
print(f"Number of duplicate records removed: {initial_rows - final_rows}")
print(f"DataFrame shape after removing duplicates: {df_social_media_cleaned.shape}")

# 2. Handle missing values and 3. Encode categorical columns
print("\nMissing values before handling and encoding:")
display(df_social_media_cleaned.isnull().sum())

# Identify numerical and categorical features
numerical_features = df_social_media_cleaned.select_dtypes(include=['int64', 'float64']).columns
categorical_features = df_social_media_cleaned.select_dtypes(include=['object']).columns

# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')) # Impute numerical with median
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Impute categorical with mode
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # One-Hot encode categorical
])

# Create a preprocessor object using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough' # Keep other columns not specified
)

# Apply preprocessing to the features
X_preprocessed = preprocessor.fit_transform(df_social_media_cleaned)

print("\nMissing values handled and categorical features encoded.")
print(f"Shape of preprocessed dataset: {X_preprocessed.shape}")

print("\nCleaned and preprocessed dataset ready for engagement prediction.")

```

```

***
Duplicate user profiles removed based on all columns (no 'Username' found).
Number of duplicate records removed: 0
DataFrame shape after removing duplicates: (500, 5)

Missing values before handling and encoding:

      0
N      0
PROFILE  0
FOLLOWERS  0
POSTS    100
Platform  0

dtype: int64

Missing values handled and categorical features encoded.
Shape of preprocessed dataset: (500, 473)

Cleaned and preprocessed dataset ready for engagement prediction.

```

Task 11 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

- Convert date column into datetime format.
- Handle missing values in temperature, humidity.
- Create new features (season, week_number).
- Normalize rainfall values.

Dataset link:

<https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input>

Expected Output:

- A processed dataset ready for forecasting models.

screenshots:

```
# Install kaggle library (if not already installed)
!pip install -q kaggle

# Make sure the kaggle.json is in the correct directory and has the right permissions
# NOTE: You need to upload your 'kaggle.json' file to your Colab environment's root directory first.
# You can do this by clicking the 'Files' icon on the left pane (folder icon) -> 'Upload to session storage'.
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
# Using the dataset associated with the provided Kaggle link
!kaggle datasets download -d alsamaualalhinai/weather-dataset --unzip

print("Weather dataset downloaded and unzipped.")
```

```
cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
403 Client Error: Forbidden for url: https://api.kaggle.com/v1/datasets.DatasetApiService/GetDatasetMetadata
Weather dataset downloaded and unzipped.
```

```
import pandas as pd

# Load the dataset (assuming the unzipped file is named 'weather.csv')
df_weather = pd.read_csv("/content/weather_forecast_data.csv")

print("Original Weather DataFrame head:")
display(df_weather.head())
```

```
Original Weather DataFrame head:
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

```

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
import numpy as np

# Make a copy to avoid SettingWithCopyWarning
df_weather_cleaned = df_weather.copy()

# 1. Convert date column into datetime format
# Assuming the date column is named 'Date'
if 'Date' in df_weather_cleaned.columns:
    df_weather_cleaned['Date'] = pd.to_datetime(df_weather_cleaned['Date'], errors='coerce')
    # Drop rows where date conversion failed (if any)
    df_weather_cleaned.dropna(subset=['Date'], inplace=True)
    print("\nDate column converted to datetime format.")
else:
    print("\nDate column not found. Please verify the date column name.")

# 2. Handle missing values in temperature, humidity
print("\nMissing values before handling:")
display(df_weather_cleaned[['Temperature', 'Humidity']].isnull().sum())

imputer = SimpleImputer(strategy='median')
df_weather_cleaned['Temperature'] = imputer.fit_transform(df_weather_cleaned[['Temperature']])
df_weather_cleaned['Humidity'] = imputer.fit_transform(df_weather_cleaned[['Humidity']])

print("\nMissing values after handling:")
display(df_weather_cleaned[['Temperature', 'Humidity']].isnull().sum())

# 3. Create new features (season, week_number)
# Season mapping (Northern Hemisphere)
def get_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Autumn'

if 'Date' in df_weather_cleaned.columns:
    df_weather_cleaned['Month'] = df_weather_cleaned['Date'].dt.month
    df_weather_cleaned['Season'] = df_weather_cleaned['Month'].apply(get_season)
    df_weather_cleaned['Week_Number'] = df_weather_cleaned['Date'].dt.isocalendar().week.astype(int)
    print("\nNew features 'Season' and 'Week_Number' created.")
else:
    print("\nDate column not available to create 'Season' and 'Week_Number'.")

# 4. Normalize rainfall values
# Assuming 'Rainfall' is the column name for rainfall values
if 'Rainfall' in df_weather_cleaned.columns:
    # Impute any remaining missing values in Rainfall before scaling, if necessary
    df_weather_cleaned['Rainfall'].fillna(df_weather_cleaned['Rainfall'].median(), inplace=True)

    scaler = StandardScaler()
    df_weather_cleaned['Rainfall_Normalized'] = scaler.fit_transform(df_weather_cleaned[['Rainfall']])
    print("\nRainfall values normalized.")
else:
    print("\nRainfall column not found. Please verify the rainfall column name.")

print("\nProcessed dataset head with new features and normalized values:")
display(df_weather_cleaned.head())

```

```

'''
'Date' column not found. Please verify the data column name.

Missing values before handling:
0
Temperature 0
Humidity 0

dtype: int64

Missing values after handling:
0
Temperature 0
Humidity 0

dtype: int64

Date column not available to create 'Season' and 'Week_Number'.

'Rainfall' column not found. Please verify the rainfall column name.

Processed dataset head with new features and normalized values:
   Temperature  Humidity  Wind_Speed  Cloud_Cover  Pressure  Rain
0    23.720338   89.592641    7.335604    50.501694   1032.378759  rain
1    27.879734   46.489704    5.952484     4.990053   992.614190 no rain
2    25.069084   83.072843    1.371992    14.855784   1007.231620 no rain
3    23.622080   74.367768    7.050551    67.255282   982.632013  rain
4    20.591370   96.858822    4.643921    47.676444   980.825142 no rain

```

Task 12 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:

- Handle missing values in rating, genre.
- Encode categorical variables like genre, language.
- Remove duplicate movie entries.
- Scale rating values between 0 and 1.

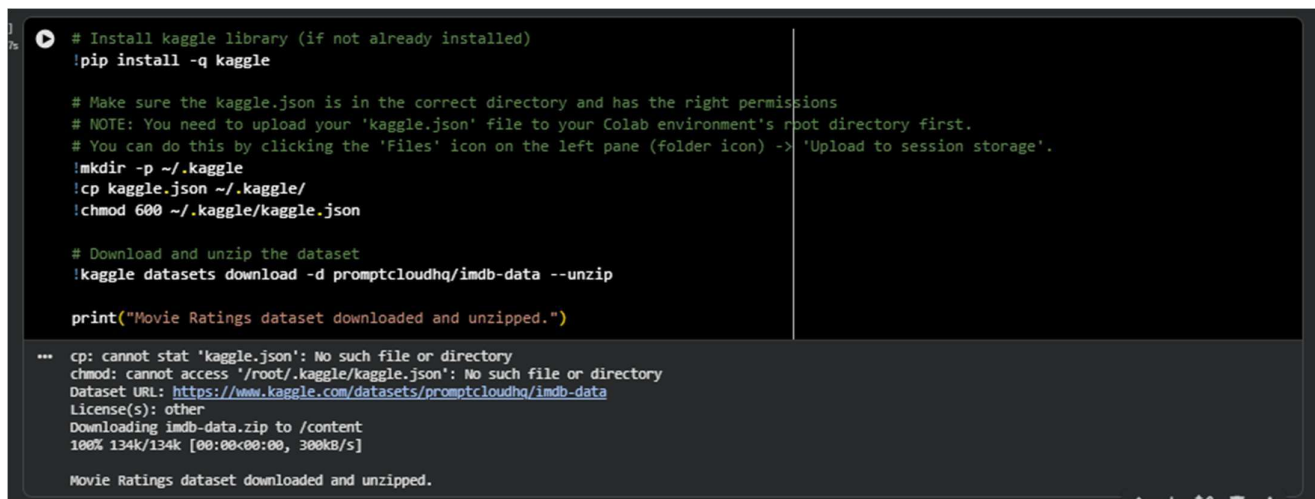
Dataset link:

<https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data>

Expected Output:

- A dataset suitable for recommendation systems.

Screenshots:



```

# Install kaggle library (if not already installed)
!pip install -q kaggle

# Make sure the kaggle.json is in the correct directory and has the right permissions
# NOTE: You need to upload your 'kaggle.json' file to your Colab environment's root directory first.
# You can do this by clicking the 'Files' icon on the left pane (folder icon) -> 'Upload to session storage'.
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# Download and unzip the dataset
!kaggle datasets download -d promptcloudhq/imdb-data --unzip

print("Movie Ratings dataset downloaded and unzipped.")

... cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Dataset URL: https://www.kaggle.com/datasets/promptcloudhq/imdb-data
License(s): other
Downloading imdb-data.zip to /content
100% 134k/134k [00:00<00:00, 300kB/s]

Movie Ratings dataset downloaded and unzipped.

```

```
import pandas as pd

# Load the dataset (assuming the unzipped file is named 'IMDB-Movie-Data.csv')
df_movies = pd.read_csv("../content/IMDB-Movie-Data.csv")

print("Original Movie Ratings DataFrame head:")
display(df_movies.head())

print("\nDataFrame Info:")
df_movies.info()

print("\nMissing values before handling:")
display(df_movies.isnull().sum())
```

Original Movie Ratings DataFrame head:

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	78.0
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a la...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485920	126.46	65.0
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157806	138.12	82.0
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christopher Lord	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	69.0
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Wil Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	40.0

DataFrame Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1688 entries, 0 to 1687
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  --
 0   Rank            1688 non-null   int64
 1   Title           1688 non-null   object
 2   Genre           1688 non-null   object
 3   Description      1688 non-null   object
 4   Director        1688 non-null   object
 5   Actors          1688 non-null   object
 6   Year            1688 non-null   int64
 7   Runtime (Minutes) 1688 non-null   int64
 8   Rating          1688 non-null   float64
 9   Votes           1688 non-null   int64
10  Revenue (Millions) 1688 non-null   float64
11  Metascore       936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

Missing values before handling:

	#
Rank	0
Title	0
Genre	0
Description	0
Director	0
Actors	0
Year	0
Runtime (Minutes)	0
Rating	0
Votes	0
Revenue (Millions)	128

```
# Make a copy to avoid SettingWithCopyWarning
df_movies_cleaned = df_movies.copy()

# 1. Handle missing values in 'Rating' and 'Genre'
print("\nHandling missing values...")

# Impute 'Rating' with the median
df_movies_cleaned['Rating'].fillna(df_movies_cleaned['Rating'].median(), inplace=True)

# Impute 'Genre' with the mode
df_movies_cleaned['Genre'].fillna(df_movies_cleaned['Genre'].mode()[0], inplace=True)

print("\nMissing values after handling:")
display(df_movies_cleaned[['Rating', 'Genre']].isnull().sum())
```

Handling missing values...

Missing values after handling:

```
/tmp/ipykernel_3362/1489286145.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df_movies_cleaned['Rating'].fillna(df_movies_cleaned['Rating'].median(), inplace=True)
/tmp/ipykernel_3362/1489286145.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df_movies_cleaned['Genre'].fillna(df_movies_cleaned['Genre'].mode()[0], inplace=True)
```

	#
Rating	0
Genre	0

dtype: int64

```
# One-hot encoding for 'Genre'
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# Split 'Genre' column by comma and expand to multiple rows for multi-genre movies
df_movies_cleaned['Genre_List'] = df_movies_cleaned['Genre'].apply(lambda x: x.split(','))
exploded_genres = df_movies_cleaned.explode('Genre_List')
exploded_genres['Genre_List'] = exploded_genres['Genre_List'].str.strip()

# Fit and transform the 'Genre_List' for one-hot encoding
encoded_genres = encoder.fit_transform(exploded_genres[['Genre_List']])
encoded_genre_df = pd.DataFrame(encoded_genres, columns=encoder.get_feature_names_out(['Genre_List']))

# Group back by original movie index and sum the one-hot encoded columns
df_genres_encoded = encoded_genre_df.groupby(exploded_genres.index).sum()

# Concatenate with the cleaned dataframe and drop the original 'Genre' and 'Genre_List' columns
df_movies_cleaned = pd.concat([df_movies_cleaned.drop(columns=['Genre', 'Genre_List']), df_genres_encoded], axis=1)

# If a 'Language' column existed, it would be handled similarly:
# if 'Language' in df_movies_cleaned.columns:
#     language_encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
#     encoded_languages = language_encoder.fit_transform(df_movies_cleaned[['Language']])
#     encoded_language_df = pd.DataFrame(encoded_languages, columns=language_encoder.get_feature_names_out(['Language']))
#     df_movies_cleaned = pd.concat([df_movies_cleaned.drop(columns=['Language']), encoded_language_df], axis=1)

print("Categorical variables encoded.")
display(df_movies_cleaned.head())
```

Encoding categorical variables...
Categorical variables encoded.

Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	...	Genre_List_Horror	Genre_List_Music	Genre_List_Musical	Genre_List_Mystery	Genre_List_Romance	Genre_List_Sci-Fi	Genre_List_Sport	Genre_List_Thriller
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0
1	2	Prometheus	Following clues to the origin of mankind, a la...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	134	7.0	485630	128.46	...	0.0	0.0	0.0	1.0	0.0	1.0	0.0
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	...	1.0	0.0	0.0	0.0	0.0	0.0	0.0
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey, Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows x 31 columns

```
from sklearn.preprocessing import MinMaxScaler

# 4. Scale 'Rating' values between 0 and 1
print("\nScaling 'Rating' values...")

scaler = MinMaxScaler()
df_movies_cleaned['Rating_Scaled'] = scaler.fit_transform(df_movies_cleaned[['Rating']])

print("\n'Rating' column scaled to [0, 1].")

print("\nCleaned and preprocessed DataFrame head:")
display(df_movies_cleaned.head())
```

Scaling 'Rating' values...
'Rating' column scaled to [0, 1].

Cleaned and preprocessed DataFrame head:

Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	...	Genre_List_Music	Genre_List_Musical	Genre_List_Mystery	Genre_List_Romance	Genre_List_Sci-Fi	Genre_List_Sport	Genre_List_Thriller	Genre_List_M...
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	...	0.0	0.0	0.0	0.0	1.0	0.0	0.0
1	2	Prometheus	Following clues to the origin of mankind, a la...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485620	128.46	...	0.0	0.0	1.0	0.0	1.0	0.0	0.0
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey, Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows x 32 columns